

YAROSLAV HAYDUK

Bachelor in Computer Science and Engineering

CHALLENGES IN LEARNING AND TEACHING SOFTWARE MODELLING: A SOCIO-TECHNICAL GROUNDED THEORY APPROACH

MASTER IN INTEGRATED MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

Draft: September 17, 2026

CHALLENGES IN LEARNING AND TEACHING SOFTWARE MODELLING: A SOCIO-TECHNICAL GROUNDED THEORY APPROACH

YAROSLAV HAYDUK

Bachelor in Computer Science and Engineering

Supervisor: Miguel Goulão

Associate Professor, NOVA University Lisbon

Co-supervisor: Grischa Liebel

Associate Professor, Reykjavik University

ABSTRACT

Software and conceptual modelling are commonly included in software engineering and related computing programmes, but they often receive less emphasis than other topics in computing education. Students frequently experience modelling as either an essential aid for understanding and communication or as burdensome extra documentation. Existing research offers valuable insights but is dispersed across modelling traditions and study contexts, with limited integrative explanation of how challenges arise and how they relate to strategies, support, and course conditions.

This thesis investigates how software modelling is experienced, taught, and learned in higher education, treating modelling as a socio-technical activity. It addresses three research questions: (RQ1) what challenges students and instructors perceive and how they manifest across interpreting, creating, refining, and evaluating models; (RQ2) which strategies and pedagogical or tool-related support needs they identify; and (RQ3) how tools, course design, domains, and feedback practices shape those experiences.

The study used Socio-Technical Grounded Theory. Fourteen semi-structured interviews (four lecturers, ten students) were open-coded into 277 codes and 78 memos, then compared into 14 categories and four phenomena. Wave 2 theoretical sampling densified properties without adding a 15th overview category. The analysis claims local saturation of that category set and theoretical sufficiency for this sample, not global theoretical saturation.

The account shows a gated first modelling move distinct from symbol overload; difficulty judging “good enough” without a compile result; late and unequally accessible feedback, with rework depending on remaining time and locked grades; demand for checks and explanations rather than generated homework; and diagrams that survive only when someone owns them and someone reads them. Course load and tool friction condition whether examples and feedback loops actually occur. Implications for teaching and educational tools are derived from these patterns but were not trialled.

Keywords: software modelling education · conceptual modelling · grounded theory · socio-technical perspective · modelling competence

RESUMO

A modelação de software e a modelação conceptual são frequentemente incluídas em cursos de Engenharia de Software e áreas relacionadas, mas tendem a receber menos destaque do que outros tópicos na educação em Computação. Os estudantes frequentemente percebem a modelação como um apoio essencial à compreensão e comunicação ou, pelo contrário, como documentação extra. A investigação existente fornece contributos relevantes, mas encontra-se distribuída por diferentes tradições de modelação, com explicação integrada ainda limitada sobre o modo como os desafios emergem e se relacionam com estratégias, apoio e condições da unidade curricular.

Esta tese investiga como a modelação de software é experienciada, ensinada e aprendida no ensino superior, tratando a modelação como uma atividade socio-técnica. O estudo é guiado por três questões: (RQ1) que desafios estudantes e docentes percebem, e como se manifestam ao interpretar, criar, refinar e avaliar modelos; (RQ2) que estratégias e necessidades de apoio (pedagógicas e de ferramentas) são identificadas; e (RQ3) de que forma ferramentas, desenho curricular, domínios e práticas de feedback moldam essas experiências.

O estudo usou Socio-Technical Grounded Theory. Catorze entrevistas semi-estruturadas (quatro docentes, dez estudantes) foram sujeitas a codificação aberta (277 códigos e 78 memorandos) e comparadas em 14 categorias e quatro fenómenos. A amostragem teórica da segunda vaga densificou propriedades sem acrescentar uma décima quinta categoria de visão geral. A análise argumenta saturação local desse conjunto de categorias e suficiência teórica para esta amostra, não saturação teórica global.

O relato mostra um primeiro passo de modelação condicionado por um caminho trabalhado, distinto da sobrecarga de símbolos; dificuldade em julgar “suficientemente bom” sem um resultado de compilação; feedback tardio e de acesso desigual, com re-trabalho dependente do tempo restante e da nota já fechada; procura de verificações e explicações em vez de trabalhos gerados automaticamente; e diagramas que só se mantêm quando alguém os assume e alguém os lê. A carga curricular e o atrito das ferramentas condicionam a existência efetiva de exemplos e ciclos de feedback. As implicações para o ensino e para ferramentas educativas derivam destes padrões e não foram ensaiadas.

Palavras-chave: educação em modelação de software · modelação conceptual · grounded theory · perspectiva socio-técnica · competência de modelação

CONTENTS

1	Introduction	1
1.1	Motivation	1
1.2	Problem statement and research gap	1
1.3	Aim and research questions	2
1.4	Research approach overview	2
1.5	Contributions	3
1.6	Thesis structure	3
2	Background	4
2.1	Software modelling & conceptual modelling	4
2.1.1	Purpose and role of modelling in software engineering	4
2.1.2	Conceptual modelling as a broader discipline	4
2.1.3	Modelling methods, languages, and artefacts	5
2.1.4	Implications for the scope of this thesis	5
2.2	Modelling education	6
2.2.1	Overview of conceptual modelling education	6
2.2.2	Bloom-based perspectives on modelling learning outcomes	6
2.2.3	Process and goal modelling education	7
2.2.4	Modelling, tools, and model-driven engineering in education	8
2.2.5	Competence-oriented and human-centred perspectives	8
2.2.6	Implications for this thesis	9
2.3	Grounded Theory	9
2.3.1	Overview and key characteristics	9
2.3.2	Grounded Theory in software engineering	10
2.3.3	Socio-Technical Grounded Theory	11
2.3.4	Implications for this thesis	11
2.4	Bloom’s Taxonomy and learning outcomes	11
2.4.1	Overview of Bloom’s revised taxonomy	11
2.4.2	Learning outcomes in computing and software engineering	12

2.4.3	Bloom-based formulations of modelling learning outcomes	12
2.4.4	Bloom, competencies, and this thesis	13
2.5	Human factors in modelling and SE education	13
2.5.1	Motivation, engagement, and the “value” of modelling	13
2.5.2	Inclusion, domain diversity, and participation barriers	14
2.5.3	Tools as mediators of learning experience	14
2.5.4	Authenticity, experiential learning, and social dynamics	15
2.5.5	Competence development and human-centred educational design	15
2.5.6	Implications for this thesis	16
3	Related Work	17
3.1	Conceptual modelling education landscape	17
3.2	Modelling frameworks	17
3.2.1	CaMeLOT: a framework for conceptual data modelling learning outcomes	17
3.2.2	Domain Modelling in Bloom: empirical analysis of assessment practices	18
3.2.3	Teaching conceptual modelling and metamodeling	18
3.3	Empirical studies on modelling learning challenges	18
3.3.1	iStar learning challenges and support requirements	18
3.3.2	Process modelling education and the need for empirical validation	19
3.4	Tools and model-driven engineering in education	19
3.4.1	Perception of tools and UML in MDE education	19
3.4.2	Human factors in model-driven engineering	20
3.5	Human factors and learning context in modelling education	20
3.5.1	Domain diversity, motivation, and inclusion	20
3.5.2	Experiential learning and authenticity	20
3.6	Competence-oriented agendas and methodological precedents	20
3.6.1	Competency identification and development in SE education	21
3.6.2	Grounded Theory in software engineering research	21
3.7	Relating the literature to the present study	21
3.7.1	How the core literature speaks to the findings	21
3.7.2	Additional work that strengthens selected claims	23
3.7.3	Where the literature remains thin	24
3.8	Summary	24
4	Methodology	25
4.1	Research design	25
4.2	Research questions and operational focus	26
4.3	Study context and participants	26
4.4	Data collection	27

4.4.1	Interview guides and pilot	27
4.4.2	Incident-based probing	28
4.4.3	Interviews	28
4.4.4	Transcription and analytic identifiers	28
4.5	Data analysis	29
4.5.1	Grounded Theory variant and socio-technical stance	29
4.5.2	Coding and memoing	29
4.5.3	Category development and the findings map	29
4.5.4	Theoretical sampling	30
4.5.5	Local saturation of the category set	31
4.5.6	Bloom’s taxonomy as a sensitising concept	31
4.6	Rigour and trustworthiness	32
4.7	Ethics and data management	33
4.8	Methodological limitations	33
4.9	Worked example of the coding process	33
5	Findings	36
5.1	How to read this chapter	36
5.2	RQ1: Challenges across modelling activities	36
5.2.1	CAT1 — Starting is gated without a worked path	37
5.2.2	CAT2 — Notation and symbol overload create doubt	38
5.2.3	CAT12 — Hard to judge good enough	39
5.2.4	CAT14 — Cross-notation switching is hard	39
5.2.5	PH1 — Turning a written problem into the first diagram	40
5.3	RQ2: Strategies and support needs	41
5.3.1	CAT3 — Teacher examples and walkthroughs unlock work	41
5.3.2	CAT4 — Students invent process strategies when teaching is thin	42
5.3.3	CAT5 — Feedback arrives too late	42
5.3.4	CAT6 — Feedback access depends on initiative and format	42
5.3.5	CAT13 — AI used as explainer, not generator	43
5.3.6	PH2 — The teacher is the only compile button	43
5.4	RQ3: Socio-technical conditions	44
5.4.1	CAT7 — Semantic checking is valued	44
5.4.2	CAT8 — Tool friction, switching, and cost	45
5.4.3	PH3 — Want checks and explanations, not auto-drawn homework	45
5.4.4	CAT9 — Groups can kill or save the model	45
5.4.5	CAT10 — Real audience makes modelling worth it	45
5.4.6	CAT11 — Course load and theory–practice gaps	47
5.4.7	PH4 — A diagram survives only if someone owns it and someone reads it	47
5.5	Category summary	48

6	Discussion	51
6.1	Answering the research questions	51
6.1.1	RQ1 — Challenges across modelling activities	51
6.1.2	RQ2 — Strategies and support needs	52
6.1.3	RQ3 — Socio-technical conditions	52
6.2	Implications	53
6.3	Limits of the claims	54
6.4	Future work	56
7	Conclusion	57
7.1	Answers to the research questions	57
7.2	Contributions	57
7.3	Limits	58
7.4	Closing	58
Appendices		
A	Pilot interview guides	59
A.1	Instructor pilot interview guide (20–30 min)	59
A.2	Student pilot interview guide (20–30 min)	62
A.3	Mapping of interview questions to research questions	64
B	Analytic audit trail	66
B.1	How to read the cards	66
B.2	Index of reproduced memos	66
B.3	Memo cards	67
B.4	Open-code exemplars	75
Bibliography		79

LIST OF FIGURES

4.1	Analytic evidence chain used in this study. Identifiers are those stored in the vault notes; they are not timestamps. Selected memos and codes are reproduced in Appendix B.	29
4.2	Theoretical sampling across two waves. Wave 2 densified properties of the existing 14-category set; it did not add a 15th overview category. The stop rule is local saturation of that set, not global theoretical saturation.	30
5.1	Findings map: research questions, overview categories, and phenomena. CAT2, CAT11, and CAT14 answer research questions but do not feed a phenomenon; they contrast or condition other categories (Figure 5.11).	38
5.2	CAT1 variation: room-level waiting, a project that required notations not yet taught, and students who can start without copying (cases L1, S3, S5, S7, S8, S10).	39
5.3	CAT1 versus CAT2. S1 could name the high-level task and still stall on notation; that incident belongs on the right, not as another start-gate.	39
5.4	CAT14 translation loss. The missing piece is often a taught bridge: what must survive when the family changes (S6, S7, S10; not S9).	40
5.5	PH1 focused view. Feeding categories are CAT1, CAT3, and CAT4; CAT2 is a contrast, not a feeder.	41
5.6	CAT5 versus CAT6. <i>When</i> comments arrive is not the same as <i>who</i> can get them (S4–S10).	43
5.7	PH2 focused view. CAT12 names the missing compile result; CAT5 and CAT6 name when and how the social substitute arrives.	44
5.8	PH3 focused view. PH2 is the person-signal they actually get; PH3 is the check and explainer they asked for. Checking is wanted (CAT7); a heavy, optional, or antique tool is still dropped (CAT8); AI is used to explain options, not to generate graded models (CAT13).	46
5.9	CAT9: groups can kill a diagram or keep it alive (S4, S7, S10; not S9).	46
5.10	PH4 focused view. CAT9 and CAT10 feed the phenomenon; CAT11 blocks the teaching loops around it.	47

5.11 All category-to-category relations from the findings map. Solid arrows: helps explain. Dashed: contrasts, blocks, interacts, substitutes, or can split. Panels are grouped by where the arrows arrive, not by a single phenomenon: load, groups, late feedback, and switching all reach the start-gate; checks, access, audience, AI, and switching all reach judging. Feeds into PH1–PH4 are in Figures 5.5–5.10. Local sufficiency of this 14-category set is argued in Chapter 4. 49

LIST OF TABLES

4.1	Interview corpus (anonymised). Context is given at a general grain.	27
4.2	First-cycle codes and memos by case.	30
4.3	Cumulative overview-category count across analytic waves.	31
4.4	Wave-2 densification of existing categories. Cells: empty; inst. (instantiates a known property); new (new property or dimension); neg. (negative or boundary case); — (not used: the interview did not supply an incident of that strength). S7/S10 switch incidents are counted under CAT14, not as extra CAT2 warrants.	32
4.5	Illustrative coding chain from two student interviews (S1, S2). Later category IDs are those used in Chapter 5.	34
5.1	Overview of the fourteen categories. Detailed evidence follows in the sections below.	37
5.2	CAT4 process tactics invented by students when a taught path was thin or late.	42
5.3	Overview categories, primary research question, one-line claim, and strongest memos.	50
6.1	Open questions that directed wave 2.	55
6.2	CAT5 rework modes after comments (wave 2). Variation is a dimension of late feedback, not a dedicated refine category.	55
6.3	CAT6 access conditions. Extra comments still tend to be asked-for; the format of the channel varies.	55
A.1	Mapping of pilot interview guide questions to research questions.	65
B.1	Memos reproduced below, by overview category (same identifiers as Table 5.3).	67

INTRODUCTION

1.1 Motivation

Software systems are complex socio-technical artefacts that must evolve under technical, organisational, and human constraints. In this setting, software and conceptual modelling provide structured abstractions that can support reasoning, communication, and early validation before implementation [5]. In higher education, modelling is therefore commonly taught in software engineering and related programmes through notations and methods such as UML, BPMN, ER, or goal-oriented requirements approaches [20, 25, 26].

Despite its recognised importance, students often perceive modelling in polarised ways: as a valuable instrument for understanding and communicating about systems, or as burdensome “extra documentation” that competes with implementation time [8, 13, 21]. These perceptions are shaped by course-level conditions and by how modelling is situated within the learning workflow—for example, the tools used, the nature of modelling tasks and domains, the role of models in assessment, and how feedback is provided [13, 21]. Understanding how such conditions relate to modelling practice and competence development is important for improving modelling education and for fostering modelling skills that transfer beyond individual course contexts [26].

1.2 Problem statement and research gap

Research on modelling education has produced valuable insights, but evidence and concepts are distributed across different modelling traditions and study contexts [26]. Prior work has (i) proposed outcome-oriented frameworks and taxonomies that help articulate modelling competencies and highlight recurring gaps (e.g., limited explicit attention to evaluation and metacognitive aspects) [2, 3, 4], and (ii) reported, in method- and course-specific empirical studies, recurring difficulties such as grasping construct meaning, moving from requirements to models, and judging model adequacy, often alongside suggestions for pedagogical or tool-mediated support [20, 25].

However, there remains a need for an integrated, empirically grounded understanding

of modelling education challenges and support needs that reflects both student and instructor viewpoints across educational contexts, and that connects reported difficulties to the socio-technical conditions in which modelling is taught and practiced [21, 26].

1.3 Aim and research questions

The aim of this thesis is to develop an empirically grounded account of modelling education challenges and support needs in higher education, treating modelling as a socio-technical activity that combines conceptual understanding, procedural strategies, and contextual influences.

The study is guided by the following research questions:

- **RQ1:** What challenges do students and instructors perceive in learning and teaching software modelling in higher education, and how do these challenges manifest across different modelling activities (e.g., interpreting, creating, refining, and evaluating models)?
- **RQ2:** What strategies do students and instructors employ to address these challenges, and what support needs (pedagogical and tool-related) do they identify?
- **RQ3:** How do socio-technical factors (e.g., tools, course design, domains, feedback practices) shape modelling learning and teaching experiences?

In line with these questions, the analysis focuses respectively on (RQ1) challenges as they appear across modelling activities, (RQ2) coping strategies and articulated support needs, and (RQ3) the socio-technical conditions that shape how modelling is learned and taught.

1.4 Research approach overview

To address these questions, the thesis adopted a qualitative, theory-building approach. The study used Grounded Theory informed by a socio-technical stance that attends to interactions between social and technical elements in modelling education [15, 30]. Data were collected from four instructors and ten students (14 interviews) through semi-structured interviews. Guides used incident probes about a recent assignment, laboratory, or text-to-model start; unlike the iStar learning study, this work did not combine a tutorial and a modelling session with a later interview [20]. Bloom’s revised taxonomy was used as a sensitising concept to help communicate the cognitive character of reported challenges without constraining the grounded analysis [18, 30]. The primary outcome is a grounded explanatory account of 14 categories and four phenomena; secondarily, the thesis derives implications for teaching practice and educational tool support, labelled as implications rather than as trialled interventions.

1.5 Contributions

This thesis contributes to software and conceptual modelling education research in four complementary ways. First, it provides an empirically grounded characterisation of the challenges that students and instructors perceive in learning and teaching modelling, including difficulties that arise across interpreting problem descriptions, constructing models, refining them under late feedback, and judging their quality.

Second, it offers an integrated account of the strategies participants use to cope with these challenges and the support needs they articulate. These support needs span pedagogical practices (e.g., scaffolding, feedback, assessment framing) and tool-mediated support (e.g., checking rather than generation, and mechanisms for managing complexity), and are derived from concrete episodes recounted in the interviews.

Third, the thesis develops a socio-technical explanatory perspective that links modelling challenges and coping strategies to contextual factors such as tool ecosystems, course design choices, feedback practices, group work, and assignment domains. This perspective clarifies how modelling experiences are shaped by interactions between cognitive demands and the educational settings in which modelling is taught and practiced.

Finally, the thesis provides evidence-informed implications for modelling education design. These implications are formulated as recommendations for instructors and course designers and as requirements-level insights for those developing educational modelling tools. They are not claimed as validated intervention effects.

1.6 Thesis structure

The remainder of this thesis is structured as follows. Chapter 2 provides background on software and conceptual modelling, modelling education, grounded theory, Bloom-based learning outcomes, and human factors relevant to modelling learning. Chapter 3 reviews related work on modelling education frameworks, empirical studies of modelling learning challenges, tooling and model-driven engineering in education, and contextual factors such as motivation, inclusion, and authenticity. Chapter 4 presents the research design, the 14-interview corpus, data collection and analysis procedures, local saturation of the category set, and considerations of rigour, ethics, and limitations. Chapter 5 reports the 14 categories and four phenomena, organised by research question. Chapter 6 interprets those findings against related work, states implications, and keeps unfulfilled open questions as future work. Chapter 7 summarises the contributions and bounds of the study.

BACKGROUND

2.1 Software modelling & conceptual modelling

2.1.1 Purpose and role of modelling in software engineering

Software modelling supports reasoning about software systems by creating representations that abstract from reality to emphasise aspects that matter for a given purpose and audience. In software engineering, models can serve multiple roles: they may act as design blueprints, facilitate communication among stakeholders, and enable analysis and validation of properties before implementation [5, 26].

A useful clarification is that modelling should not be reduced to producing diagrammatic pictures. Rather, models are structured artefacts expressed in a modelling language with defined meaning, and their usefulness depends on what they represent and how they are intended to be used. Kühne highlights that the term *model* is often used ambiguously, and proposes a distinction between *type models* (the modelling languages or model types that define what can be expressed) and *token models* (the concrete model instances created for a particular problem) [19]. This distinction helps explain why learning modelling involves more than symbol recognition: it requires understanding the language-level concepts and constraints (type level) and being able to construct, adapt, and assess concrete models for specific tasks (token level).

2.1.2 Conceptual modelling as a broader discipline

It is useful to distinguish between software modelling as a practice within software engineering and conceptual modelling as a broader discipline concerned with representing, structuring, and reasoning about domains and systems across fields. Conceptual modelling is not limited to software development alone; it is also used in organisational analysis, enterprise and service design, data-intensive systems, and other domains where abstraction is needed to manage complexity and support understanding [26]. Reviews of conceptual modelling education underline its interdisciplinary nature and identify a broad

family of modelling subtypes—including data, process, goal, and enterprise modelling—that often evolve in parallel, with limited pedagogical or theoretical integration across subfields [2, 3, 26].

This disciplinary perspective is particularly relevant for research on teaching and learning software modelling. When modelling is treated primarily as a supplementary topic within software engineering courses, students may not fully appreciate its significance and may develop oversimplified views of its purpose and applicability. In contrast, the conceptual modelling perspective frames modelling languages as knowledge schemas: they define what can be expressed, how meaning is structured, and what kinds of analysis or tooling become possible. Understanding modelling therefore requires engaging with notational elements and underlying semantics, as well as modelling goals and the intended audience of the produced model artefacts [5].

2.1.3 Modelling methods, languages, and artefacts

Several authors emphasise that a modelling method is more than a set of symbols or a notation. A modelling method can be understood as comprising a modelling language (including syntax, semantics, and notation), a modelling procedure (the process or steps to create models), and supporting mechanisms and algorithms such as analysis, simulation, or transformation capabilities [4, 5]. This holistic view clarifies why students may be able to reproduce diagrammatic symbols yet still struggle to apply modelling meaningfully: the procedural and analytical aspects of modelling are often less explicitly taught or assessed.

From this viewpoint, the value of a modelling language is not only in enabling representation, but also in supporting reasoning. When a language is treated as a structured schema, models become more than static diagrams: they can become queryable, analysable, and transformable artefacts that enable deeper engagement with a domain. This also motivates tool support that can provide feedback on syntactic correctness, highlight semantic issues, help manage complexity, or automate parts of modelling tasks [2, 26].

2.1.4 Implications for the scope of this thesis

This thesis adopts the view that software modelling is an instantiation of the broader discipline of conceptual modelling. It therefore treats modelling as a discipline involving conceptual understanding of modelling constructs and their semantics, procedural competence in constructing and refining model artefacts from problem descriptions, analytical judgement of model quality and fit-for-purpose, and socio-technical awareness of how modelling practices are shaped by tools, learning contexts, and stakeholder needs. This framing aligns with work on modelling learning outcomes and competencies, which emphasises not only notational knowledge but also procedural skills, evaluative abilities, and reflective understanding [2, 3, 4, 27]. This perspective motivates the subsequent background on modelling education, grounded theory, and learning outcomes, and provides

a foundation for investigating modelling challenges from both student and instructor perspectives.

2.2 Modelling education

2.2.1 Overview of conceptual modelling education

Conceptual and software modelling are widely taught across computing and software-related programmes, but they are taught in diverse ways and appear in different course contexts and curricular roles. Maslov et al. [26] map the conceptual modelling education literature and show that educational work spans multiple modelling sub-communities (e.g., data modelling, process modelling, enterprise modelling), which have developed partially independent emphases, terminology, and pedagogical traditions [26]. Within this landscape, modelling appears in contexts ranging from introductory courses to advanced software engineering modules, and courses emphasise different modelling families and purposes (e.g., data-centric, process-centric, goal-oriented).

The same studies also highlight that much practice in modelling education is shaped by instructor experience and local institutional constraints, as well as by the availability of tools, rather than by systematically articulated educational frameworks or empirically validated design principles [26]. While there is broad consensus on the importance of modelling skills for software professionals, there is less agreement on which specific *competencies* learners should acquire, how these competencies should be scaffolded, and how learning should be assessed in a reliable and educationally meaningful way.

2.2.2 Bloom-based perspectives on modelling learning outcomes

To make modelling learning outcomes more explicit and comparable across courses and contexts, several authors have proposed adaptations of Bloom’s revised taxonomy for conceptual and domain modelling. Starting from Bloom’s six cognitive process categories—*remember, understand, apply, analyse, evaluate, and create* [18]—these works operationalise them in modelling terms. Typical modelling-specific interpretations include recalling notation elements and rules (remember), explaining construct meaning and purpose (understand), constructing models by applying modelling rules and procedures to a problem description (apply), checking models for consistency/completeness and identifying defects (analyse), judging model quality and comparing alternatives against criteria (evaluate), and producing or extending models and modelling solutions for new situations (create).

Bogdanova and Snoeck analyse learning outcomes and assessment tasks for domain modelling through the lens of Bloom’s revised taxonomy, reporting that many tasks concentrate on *understanding, analysing, and creating conceptual knowledge*, while giving less explicit attention to *evaluation* activities and to *procedural* and *metacognitive* knowledge [3]. Notably, across Bloom-informed modelling frameworks the mapping between

Bloom categories, modelling tasks, and knowledge types is not uniform (some works emphasise cognitive processes, others additionally foreground the knowledge dimension); however, a recurring observation is the relative underrepresentation of evaluation and reflective/metacognitive aspects in current educational practice.

Bork extends this Bloom-based perspective to both conceptual modelling and meta-modeling, mapping knowledge and cognitive dimensions to all components of a modelling method (language, semantics, notation, procedure, mechanisms and algorithms) [4]. An evaluation of a “Smart City” teaching case using this framework reveals that even rich, practice-oriented material may still underemphasise certain cognitive levels (e.g., evaluation) or knowledge types (e.g., metacognitive knowledge). These works collectively suggest that Bloom’s taxonomy can provide a useful lens for reflecting on and improving modelling education, by making learning objectives more explicit and balanced across different dimensions.

2.2.3 Process and goal modelling education

Beyond data and structural modelling, there is growing interest in how students learn process and goal-oriented modelling. For business process modelling with BPMN, Maslov outlines a research agenda for developing empirically validated process-modelling education that integrates insights from conceptual modelling education, process model quality research, and learning sciences [25]. This agenda highlights the need for systematic mapping of existing work, empirical studies of novice modellers’ behaviour and typical errors, and the design and experimental evaluation of BPMN-specific teaching strategies and feedback mechanisms. This reflects a broader recognition that teaching process modelling effectively requires more than just introducing notation; it requires attention to cognitive demands, quality criteria, and the kinds of feedback needed to help learners improve.

Goal-oriented requirements modelling approaches introduce additional layers of complexity, as it requires learners to reason about stakeholder intentions, rationales, and dependencies. An iStar learning study explicitly investigates the challenges students face when learning iStar, using a combination of Bloom-based task framing and grounded theory analysis of tutorials, modelling sessions, and interviews [20]. The authors report difficulties at multiple cognitive levels, including understanding the semantics of iStar constructs, applying the notation to real requirements, and analysing models to assess completeness and appropriateness. They derive a catalogue of strategies and tool-supported requirements (e.g., conceptual clarification, syntax checking, process guidance, templates, visualisation, and semi-automation modelling from text) that could help address these challenges [20].

These strands of work on BPMN and iStar education illustrate that different modelling families pose distinct learning challenges and may require specialised pedagogical approaches. They also reinforce broader themes in modelling education, including the need for scaffolding from text to model, support for managing complexity, and explicit

attention to how learners evaluate and refine their models.

2.2.4 Modelling, tools, and model-driven engineering in education

Modelling in educational settings is often mediated by tools, ranging from simple drawing tools and web-based modellers to industrial-grade model-driven engineering (MDE) environments. Tool choice can influence students' perceptions of modelling and their opportunities to practice particular skills. Liebel et al. conducted a multi-case study of model-driven software engineering courses that use different tools (e.g., Umple, Papyrus) and place different emphasis on model-driven architecture and code generation [21]. Their results indicate that students' perceptions of tools are highly context-dependent: the same tool may be perceived positively in one course setup and negatively in another, depending on project design, available support, and the roles that models play (e.g., for communication versus strict code generation). They also find that perceptions of UML as modelling language are more positive and robust when models are used for requirements and analysis, rather than only as precise inputs to code generators [21].

These findings align with broader discussions of human factors in model-driven engineering, which argue that educational tools should prioritise usability, cognitive support, constructive feedback, and alignment with learning goals over industrial feature completeness [22]. From an educational perspective, this suggests that tool selection and integration should be carefully considered to ensure they support the intended modelling competencies and do not inadvertently increase cognitive load or distract from learning objectives.

2.2.5 Competence-oriented and human-centred perspectives

Broader software engineering education research brings in complementary competence-oriented and human-centred perspectives that can inform modelling education. Sedelmaier and Landes propose the EVELIN research agenda for identifying and developing required competencies in software engineering, arguing that curricula should be grounded in explicit competence models rather than only topic lists [27]. They introduce a six-level classification system (remember, understand, explain, use, apply, develop) for technical competencies and outline an iterative, data-driven approach to relating teaching methods and structural conditions to competency development [27]. This approach resonates with the Bloom-based frameworks discussed earlier, but emphasises the need for empirical validation and alignment with real-world professional requirements.

Work on human factors in modelling and software engineering education further highlights the role of motivation, inclusion, and authenticity. For example, studies on domain diversity and motivation in modelling courses show that educators' assumptions about which domains are motivating (e.g., games, automation) do not always match students' preferences, and that students value domains that connect to their interests and everyday or socially relevant contexts [13]. Students also report that feeling excluded by

unfamiliar or culturally distant domains, or by assignment setups that do not fit their backgrounds, can hinder their engagement and learning [13]. More generally, experiential education initiatives emphasise the value of working on real systems, real tasks, and with real mentors to bridge the gap between classroom modelling exercises and the complexities of practice [16].

Taken together, these competence-oriented and human-centred perspectives reinforce the idea that modelling education should be designed not only around notations and tools, but also around the competencies to be developed, the diversity of learners, and the authenticity and inclusiveness of modelling tasks and domains.

2.2.6 Implications for this thesis

The literature on modelling education, Bloom-based frameworks, process and goal modelling, tooling, competencies, and human factors provides several important implications for this thesis. First, it suggests that modelling learning outcomes can be systematically described in terms of cognitive and knowledge dimensions, and that existing courses may underemphasise evaluation and metacognitive aspects of modelling competence [2, 3, 4]. Second, it shows that different modelling families (e.g., data, process, goal) can pose partly distinct learning demands and error patterns, meaning that findings are not automatically transferable across notations and tasks. At the same time, family-specific empirical studies—such as the iStar learning work—show how challenges can be elicited from a concrete modelling activity and how support requirements can be derived from learner experience [20, 25]. This thesis takes that interest in situated accounts into a broader educational scope, but uses incident probes inside interviews rather than a tutorial-plus-modelling session. Third, it highlights that tools and model-driven approaches can both enable and hinder learning, depending on how they are integrated into course designs and on the kind of feedback and affordances they provide [21, 22]. Finally, competence-oriented and human-centred research indicates that modelling education must be sensitive to students' motivations, backgrounds, and learning environments, and that explicit competence models can support more systematic design and evaluation of teaching [8, 16, 27].

These insights inform the research questions and methodological choices of this thesis, which aims to investigate how students and teachers experience modelling, which challenges they perceive, and what kinds of support may foster the development of modelling competencies in realistic educational contexts.

2.3 Grounded Theory

2.3.1 Overview and key characteristics

Grounded Theory (GT) is a qualitative research methodology originally developed in sociology with the aim of generating theory that is systematically grounded in data

rather than deduced from prior hypotheses. Instead of starting from a fixed theoretical framework, GT proceeds iteratively: data collection and analysis occur simultaneously, and emerging insights guide subsequent sampling and questioning. Core practices include *coding* (assigning conceptual labels to segments of data), *constant comparison* (comparing new data with existing codes and categories), and *theoretical sampling* (collecting new data to refine or challenge emerging concepts), and extensive *memoing* to capture analytic ideas [1, 30].

The outcome of a GT study is not a set of descriptive themes, but a more abstract, integrated explanation of how participants handle a central problem or concern. This explanation is typically organised around a *core category* that captures a dominant pattern in the data and links other categories together. Evaluation criteria emphasise the *fit* of concepts to the data, the *workability* of the emerging theory in explaining the studied phenomenon, its *relevance* to practitioners and researchers, and its *modifiability* in light of new data, rather than statistical notions of validity [1].

Over time, GT has evolved into several variants, including *Classic GT* (Glaser and Strauss), *Straussian GT* (Strauss and Corbin), and *Constructivist GT* (Charmaz) [9, 12, 31]. While these variants share core principles, they differ in the degree to which they prescribe specific coding procedures, the role of prior theory, and the epistemological stance of the researcher. For software engineering research, this diversity offers flexibility but also introduces risks of inconsistent or superficial use if methodological choices are not made explicit [30].

2.3.2 Grounded Theory in software engineering

In software engineering (SE), GT has been used to study topics that are difficult to capture through controlled experiments or surveys, such as developer experience, team practices, decision-making processes, and socio-technical phenomena. A critical review by Stol et al. [30] finds that GT has become increasingly popular in SE research, but that many studies lack methodological rigour, adopt only parts of GT procedures, and show limited reflexivity regarding the researcher's role [30]. To address these issues, they offer a set of guidelines on when GT is appropriate (e.g., for under-theorised, practice-oriented problems), how to choose among GT variants, and how to report GT studies in a way that allows readers to assess their quality.

Adolph et al. provide a concrete example of applying classical (Glaserian) GT to study how software practitioners cope with the process of development, while also reflecting on the practical challenges of conducting GT research in SE contexts [1]. They describe in detail how they conducted open coding on interviews and field observations, developed concepts and categories through constant comparison, and iteratively refined their sampling strategy to explore interesting phenomena in more depth. Their work highlights the importance of staying close to the data, avoiding overly mechanical use of qualitative analysis tools, and systematically documenting analytic decisions through

memos.

These contributions establish GT as a suitable methodology for SE when the research goal is to generate mid-level substantive theory about human and organisational aspects of software development, and they provide concrete guidance for designing and reporting such studies.

2.3.3 Socio-Technical Grounded Theory

Hoda et al. propose Socio-Technical Grounded Theory (STGT) as a specialisation of GT for SE that explicitly foregrounds the socio-technical nature of software development [15]. STGT retains the core principles of GT (iterative data collection and analysis, constant comparison, theoretical sampling, memoing) but emphasises the need to attend simultaneously to social and technical elements and their interactions. This involves, for example, paying attention to roles, norms, tools, artefacts, and organisational structures, and analysing how these co-evolve over time.

STGT also articulates expectations for rigour in terms of credibility, transferability, dependability, and confirmability, and provides detailed guidance on reporting, including the description of research context, participants, data sources, coding procedures, and the evolution of categories and core concepts [15]. Therefore, STGT offers SE researchers a reporting template tailored to the realities of socio-technical systems.

2.3.4 Implications for this thesis

This thesis investigates how students and teachers experience software and conceptual modelling, and how they perceive challenges, learning processes, and support needs. These are socio-technical questions that involve individual cognition, educational practices, tools, and organisational constraints, and they are not yet well theorised in the software modelling education literature. Grounded Theory is therefore an appropriate methodological choice for this work.

2.4 Bloom's Taxonomy and learning outcomes

2.4.1 Overview of Bloom's revised taxonomy

The original Bloom's taxonomy was presented as a hierarchical classification of educational objectives, describing increasingly complex cognitive activities from simple recall to higher-order thinking. Krathwohl's revision restructures the taxonomy into a two-dimensional framework that distinguishes between a *knowledge dimension* (factual, conceptual, procedural, metacognitive) and a *cognitive process dimension* (remember, understand, apply, analyse, evaluate, create) [18]. This revised taxonomy encourages educators to formulate learning outcomes that are explicit both about the type of knowledge involved and the level of cognitive processing expected from learners.

In the revised framework, *factual knowledge* concerns basic elements that students must know, such as terminology and specific details; *conceptual knowledge* involves relationships among elements and larger structures; *procedural knowledge* relates to methods, techniques, and criteria for using procedures; and *metacognitive knowledge* refers to awareness of one's own cognition and learning strategies. The cognitive process dimension ranges from lower-order processes such as remembering and understanding, through applying and analysing, to higher-order processes of evaluating and creating [18]. The combination of these two dimensions supports a more fine-grained articulation of what learners should know and be able to do.

2.4.2 Learning outcomes in computing and software engineering

In computing and software engineering education, Bloom's revised taxonomy is widely used to express learning outcomes and to align teaching and assessment with intended levels of cognitive demand [7, 18]. Typical applications include mapping course objectives to cognition levels, ensuring that programmes do not over-concentrate on lower-order outcomes, and checking that assessment tasks actually require the intended level of performance. However, generic formulations of the taxonomy can be difficult to apply directly in specialised domains such as software modelling, where domain-specific activities (e.g., interpreting a diagram, constructing a model from text, critiquing a modelling method) require more tailored descriptions.

This has led to several domain-specific adaptations of Bloom's taxonomy in areas such as domain modelling, conceptual data modelling, and metamodeling [2, 3, 4]. These adaptations retain the underlying cognitive-process levels while refining the knowledge categories and example tasks to reflect the particular demands of modelling work.

2.4.3 Bloom-based formulations of modelling learning outcomes

Several works in the modelling education literature explicitly build on Bloom's revised taxonomy to describe and analyse modelling-related learning outcomes. Bogdanova and Snoeck adapt the taxonomy to domain modelling by specifying what each cognitive level can mean in this context, such as remembering notation elements, understanding the meaning of relationships, applying modelling rules to construct diagrams, analysing models for completeness and consistency, evaluating alternative designs, and creating new models from complex descriptions [3]. Using this adapted taxonomy, they classify a large set of assessment tasks and show that existing practice tends to focus on understanding, analysing, and creating conceptual knowledge, with comparatively little emphasis on evaluation tasks, procedural knowledge, or metacognitive learning outcomes [3].

Extending this work, CaMeLOT proposes a three-dimensional framework for conceptual data modelling that explicitly combines Bloom's cognitive levels with the knowledge dimension and specific content areas such as classes, relationships, constraints, modelling methods, quality criteria, and tool usage [2]. The authors instantiate this framework

with example learning outcomes, demonstrating how it can be used to design, review, or compare modelling courses. Their analysis also suggests that metacognitive outcomes (e.g., reflecting on modelling strategies, recognising one’s own limitations) are rarely made explicit in current curricula [2].

Bork further generalises the Bloom-based perspective by developing separate yet related taxonomies for conceptual modelling and metamodeling [4]. He maps the knowledge and cognitive dimensions to the components of modelling methods (language, semantics, notation, procedure, mechanisms and algorithms) and uses this mapping to evaluate a Smart City teaching case. The evaluation reveals an imbalance: many activities target creation and application at higher cognitive levels, while explicit evaluate-level tasks and metacognitive activities are largely absent [4]. This pattern is consistent with the findings of Bogdanova and Snoeck and suggests that modelling education often emphasises the ability to produce models but gives less structured attention to helping students judge model quality or reflect on their modelling practice.

These Bloom-based formulations show that learning outcomes for modelling can be described systematically across different levels and types of knowledge, and they provide concrete examples of how to manage this in educational settings. They also highlight recurring gaps, particularly in relation to evaluative and metacognitive aspects of modelling competence.

2.4.4 Bloom, competencies, and this thesis

For this thesis, Bloom’s revised taxonomy and its modelling-specific adaptations provide a conceptual vocabulary for describing modelling competence in terms of cognitive demands and knowledge types. In particular, they help distinguish challenges related to understanding modelling constructs and their semantics, applying modelling procedures to construct and refine model artefacts, and exercising evaluative judgement when assessing model quality and fit-for-purpose. The recurring gaps identified in prior work—especially the limited explicit emphasis on evaluation and metacognitive outcomes in many course designs—motivate a closer empirical examination of how students and teachers experience modelling and what kinds of support they perceive as needed [2, 3, 4, 27].

2.5 Human factors in modelling and SE education

2.5.1 Motivation, engagement, and the “value” of modelling

Software and conceptual modelling are often perceived by students as either highly valuable (e.g., for understanding complex systems and communicating design intent) or as burdensome “extra documentation” that competes with implementation time [13]. Such perceptions are not merely a matter of individual preference; they reflect a broader set of human factors that shape engagement, learning effort, and ultimately competence

development. In modelling education, learners frequently report that motivation depends on whether tasks feel authentic, whether modelling outputs matter beyond grading, and whether modelling is integrated into a meaningful workflow rather than treated as a stand-alone academic exercise. [13, 16]

Research on modelling education explicitly highlights motivation as a key factor. A study on domain diversity, motivation, and inclusion in software modelling education shows that educators' assumptions about which domains are motivating do not always match students' preferences [13]. Students tend to value domains that connect to their interests, everyday experiences, or socially relevant contexts, and they report higher motivation when they have some agency (e.g., choice or personalisation of the domain) [13]. These results suggest that human factors in modelling education cannot be separated from how modelling tasks are framed, which domains they use, and whether students can see a clear purpose for modelling beyond producing a diagram.

2.5.2 Inclusion, domain diversity, and participation barriers

Human factors also include issues of inclusion and participation. Even when modelling topics are formally accessible, examples and assignments can implicitly privilege specific backgrounds, interests, or prior experiences. Domain-diversity research indicates that certain commonly used modelling domains (e.g., video games or specific technical automation scenarios) may be engaging for some students but alienating for others [13]. Students may also experience barriers related to group-work structures, competitive gamification elements, or assumptions about familiarity with particular cultural or socioeconomic contexts. These factors can influence not only motivation but also opportunities for practice and feedback, thereby affecting learning outcomes.

From a course-design perspective, these findings motivate the careful selection of modelling domains and assignment setups, balancing instructors' preferences and curricular relevance with the diversity of student experiences and interests. Providing alternative domains, allowing domain choice, and ensuring that examples do not rely on narrow cultural references can be considered human-centred strategies to improve participation and reduce disengagement [13].

2.5.3 Tools as mediators of learning experience

In modelling education, tools play a critical role as mediators of the learning experience. They actively shape how students interact with modelling concepts, including how easily they can experiment, receive feedback, manage complexity, and recover from errors. Usability issues, steep learning curves, or misalignment between tool features and learning objectives can lead to frustration and negative attitudes toward modelling, even when the underlying notation is conceptually useful.

Liebel et al. examine model-driven software engineering education through a multi-case study and report that students' perceptions of modelling tools are highly context-dependent [21]. The same tool may be perceived as useful or cumbersome depending on how it is integrated into the course design, what the modelling artefacts are used for (e.g., communication vs. code generation), and the level of support provided. They also find that students tend to have more positive perceptions of UML when models are used for analysis and communication, even if tool-related issues reduce satisfaction in code-generation contexts [21]. This suggests that negative experiences are frequently tied to the socio-technical configuration of the course (tool, task, expectations) rather than to modelling as a concept.

2.5.4 Authenticity, experiential learning, and social dynamics

Human factors in SE education also encompass the extent to which learning activities resemble real-world practice. Experiential education initiatives emphasise that authenticity can enhance motivation, engagement, strengthen soft-skill development, and improve students' understanding of the complexities of software practice. Holmes et al. identify three "dimensions of experientialism" in a large, distributed capstone programme: working on real projects (existing open-source systems and codebases), performing real tasks (non-trivial contributions), and engaging with real mentors (experienced developers embedded in the project) [16]. Students report that this combination helps them foster communication skills, teamwork, time management, and practical problem-solving abilities, while also deepening their technical understanding in authentic contexts [16]. Although modelling courses may not always be structured as capstones, the experientialism perspective is still relevant.

2.5.5 Competence development and human-centred educational design

Competence-oriented research in SE education argues that curricula should be grounded in explicit competence models of what students should be able to do, and that teaching methods and structural conditions should be linked to competence outcomes rather than treated as ad hoc choices [27]. Sedelmaier and Landes propose the EVELIN research agenda for identifying and developing required competencies in software engineering, introducing a multi-level classification system for technical competencies intended to describe progression from basic familiarity to the ability to develop new solutions [27]. This approach emphasises iterative improvement through measuring competence levels, identifying influencing factors (structural conditions and process variables), and refining course designs accordingly [27].

Adopting such a competence-oriented and human-centred perspective in modelling education may require more than selecting an engaging domain or user-friendly tool; it may also involve designing learning activities and assessments that support the systematic progression across competencies, including conceptual understanding, procedural ability

to create and refine models, evaluative judgement of model quality, and reflective awareness of modelling choices and limitations. The reviewed literature indicates evaluation and metacognitive competencies are often underemphasised in current modelling education [3, 2, 4], and human factors research provides a complementary explanation: learners may disengage when feedback is insufficient, tasks feel artificial, tools are frustrating, or domains do not resonate with their interests and experiences.

2.5.6 Implications for this thesis

The human factors literature motivates this thesis in several ways. First, it highlights that modelling education should be analysed not only in terms of notations and learning outcomes, but also in terms of student and teacher experiences shaped by motivation, inclusion, feedback, and perceived authenticity [13, 16]. Second, modelling tools should be treated as socio-technical mediators that influence cognitive load, engagement, and attitudes toward modelling, making tool choice and integration important pedagogical determinants of learning experience [21, 22]. Third, competence-oriented perspectives suggest that empirical investigation should seek to connect perceived challenges and support needs to explicit competence development and course design choices [27].

RELATED WORK

3.1 Conceptual modelling education landscape

Research on conceptual modelling education spans multiple modelling families and research communities, including data modelling, process modelling, requirements-related modelling, and model-driven development. Maslov et al. map the conceptual modelling education literature and report that educational work is distributed across multiple communities and modelling subfields that have developed partially independent terminology, learning-theory grounding, and evaluation approaches [26]. As a result, findings are often difficult to compare across modelling families, and evidence about effective teaching strategies and supports accumulates unevenly across subdomains.

This thesis builds on the landscape evidence by drawing together frameworks and empirical results from different modelling-education strands (conceptual/data, process, goal-oriented, and tool-supported modelling) and by focusing on the challenges and support needs reported by both students and instructors.

3.2 Modelling frameworks

A common approach to strengthening modelling education research is to define explicit frameworks for learning outcomes, competencies, and assessment design. Several works adapt Bloom's revised taxonomy [18] to modelling activities in order to clarify what learners should know and be able to do beyond learning a notation. The following subsections summarise three modelling-education frameworks that are especially relevant to this thesis.

3.2.1 CaMeLOT: a framework for conceptual data modelling learning outcomes

CaMeLOT proposes an educational framework for conceptual data modelling that combines Bloom's cognitive levels with a knowledge dimension (factual, conceptual, procedural, metacognitive) and modelling content areas [2]. The framework is instantiated with

detailed example learning outcomes and can be used to design course objectives, learning activities, and assessments in a systematic manner. For this thesis, CaMeLOT is valuable as a template for articulating modelling competence across cognitive and knowledge dimensions, and for motivating the inclusion of procedural and reflective elements rather than focusing exclusively on diagram production.

3.2.2 Domain Modelling in Bloom: empirical analysis of assessment practices

Bogdanova and Snoeck empirically analyse domain-modelling assessment tasks through the lens of Bloom's revised taxonomy [3]. They report that many tasks concentrate on understanding, analysing, and creating conceptual knowledge, while evaluation, procedural knowledge, and metacognitive learning outcomes receive limited explicit attention. This work provides evidence that modelling courses may implicitly expect higher-level modelling competence without systematically scaffolding or assessing foundational skills. For this thesis, these findings support examining modelling challenges in relation to cognitive demands and considering whether educational support should target not only syntax and notation use but also strategies for evaluation and refinement.

3.2.3 Teaching conceptual modelling and metamodeling

Bork extends Bloom-based framing to both conceptual modelling and metamodeling by relating cognitive and knowledge dimensions to components of modelling methods (language, semantics, notation, procedure, mechanisms and algorithms) [4]. By analysing a teaching case using the proposed framework, the work suggests that even practice-oriented materials may emphasise model creation while providing limited explicit support for evaluation tasks and metacognitive reflection. This perspective is useful for the present thesis because it frames modelling competence as method-level understanding and reflective judgement, aligning with the thesis focus on the transition from notation use to meaningful modelling practice.

3.3 Empirical studies on modelling learning challenges

Frameworks clarify educational targets, but empirical studies are needed to understand where learners struggle in practice and which supports are perceived as valuable. Two complementary strands are particularly relevant: studies of goal-oriented modelling learning challenges and research agendas for process modelling education.

3.3.1 iStar learning challenges and support requirements

A key empirical reference for this thesis is an iStar learning study, which investigates challenges beginners face when learning and practising iStar and derives requirements for facilitating learning [20]. Using Bloom-based task framing and grounded theory analysis,

the study reports difficulties at multiple cognitive levels, including understanding the semantics of modelling constructs, applying the notation to requirements scenarios, and analysing models for quality and completeness. The authors derive a catalogue of strategies and tool-supported requirements (e.g., conceptual clarification, syntax checking, process guidance, templates, advanced visualisation, and partial automation from textual requirements) and implement a subset of these requirements in a prototype tool [20]. This thesis builds on these insights by extending the focus from a single modelling method to broader modelling education contexts and by eliciting perspectives from both students and instructors.

3.3.2 Process modelling education and the need for empirical validation

In the domain of process modelling, Maslov argues for empirically validated process-modelling education using BPMN and outlines a research agenda that integrates conceptual modelling education, process model quality research, and learning theory [25]. The agenda emphasises systematic synthesis of existing work, empirical investigation of novice modelling behaviour and typical errors, and the design and experimental validation of BPMN-specific teaching strategies and feedback mechanisms. This agenda is relevant to the present thesis as a parallel strand: it highlights that modelling families pose distinct cognitive and practical demands while pointing to recurring educational needs such as scaffolding, feedback, and support for managing modelling complexity.

3.4 Tools and model-driven engineering in education

Modelling education is strongly shaped by tool choice, especially in model-driven and model-based engineering settings. Tools can support learners through feedback, automation, and manageability, but they can also introduce friction that influences how students perceive modelling tasks and their own competence.

3.4.1 Perception of tools and UML in MDE education

Liebel et al. show that students' tool experiences depend less on the tool "in isolation" than on how it is embedded in the course—including the support provided, the surrounding workflow, and the expectations placed on models (for example, exploratory communication artefacts versus precise inputs to downstream automation) [21]. The study also suggests that perceptions of UML are more robustly positive when models are used for requirements and analysis rather than primarily as code generation inputs [21]. These results motivate treating tooling and course design as socio-technical factors that shape modelling experiences, rather than as neutral delivery mechanisms.

3.4.2 Human factors in model-driven engineering

Work on human factors in model-driven engineering argues for research that accounts for usability, cognitive load, and the alignment between tools and user goals [22]. From an educational perspective, this supports the view that modelling tools should be evaluated not only by feature completeness, but also by how they support learning processes such as exploration, error detection, reflection, and progressive mastery.

3.5 Human factors and learning context in modelling education

Beyond frameworks, methods, and tools, modelling education is influenced by motivational and contextual factors such as domain choice, inclusiveness, and the authenticity of tasks.

3.5.1 Domain diversity, motivation, and inclusion

Research on domain diversity and motivation in modelling education reports that educators' assumptions about "motivating" domains do not always match students' preferences, and that students value domains that connect to their interests and everyday or socially relevant contexts [13]. The work also highlights that assignment design can unintentionally exclude learners when domains or examples presume cultural, economic, or experiential background that not all students share [13]. These findings motivate treating domain choice and inclusiveness as relevant contextual variables when interpreting modelling learning experiences.

3.5.2 Experiential learning and authenticity

Holmes et al. propose dimensions of experientialism in software engineering education and show that students value learning experiences that involve real projects, real tasks, and real mentors [16]. Although this work is not modelling-specific, it contributes a useful lens for interpreting how authenticity and mentorship can influence learners' engagement and competence development, and it supports the relevance of studying modelling experiences in realistic educational contexts.

3.6 Competence-oriented agendas and methodological precedents

Competence-oriented perspectives provide a complementary lens for structuring educational goals and for linking teaching approaches to measurable outcomes.

3.6.1 Competency identification and development in SE education

Sedelmaier and Landes propose the EVELIN research agenda for identifying and developing competencies in software engineering and argue that curricula should be grounded in explicit competence models rather than topic lists [27]. They introduce a six-level classification system for technical competencies and advocate an iterative approach in which competence targets are identified, competence levels are measured, and teaching strategies and structural constraints are adjusted over time [27]. While not specific to modelling, this agenda supports treating modelling competence as multi-dimensional (conceptual, procedural, analytical, socio-technical) and complements Bloom-based modelling frameworks by emphasising explicit competence models and iterative alignment between competence targets, teaching methods, and structural constraints.

3.6.2 Grounded Theory in software engineering research

Methodologically, grounded theory has been widely discussed in software engineering research as an approach for developing explanatory theory from qualitative data. Stol et al. provide a critical review and practical guidelines for conducting grounded theory in software engineering [30], while Hoda et al. propose a socio-technical grounded theory perspective that explicitly accounts for the interplay between social and technical factors [15]. Adolph et al. offer a practical account of using grounded theory to study the experience of software development [1]. These works provide methodological grounding for the approach used in this thesis and support the use of interviews and iterative coding to derive an explanatory account from participant experiences.

3.7 Relating the literature to the present study

This section places the literature above, and a small set of additional papers, against the 14-category account in Chapter 5. The additional papers were read because they speak to claims that the core set covers only thinly—delayed critique of student diagrams, group versus individual UML work, consistency across modelling views, and the reliability of LLM-generated UML. They strengthen selected claims; they do not replace the core literature.

3.7.1 How the core literature speaks to the findings

Maslov et al.’s picture of conceptual modelling education as fragmented across families [26] is the landscape condition under which the present corpus was collected: interviews span UML, BPMN, ER, iStar, and courses that teach several of those notations together, rather than a single-language laboratory. The same fragmentation is the practical cost later labelled CAT14.

The Bloom-informed frameworks [2, 3, 4] name an evaluation and metacognitive gap in intended learning outcomes. That vocabulary is useful for CAT12 and PH2: students invent checklists and wait for the teacher because a compile-like signal is missing. The frameworks describe intended learning outcomes. They do not, by themselves, describe the lived substitutes observed in the interviews, and this study did not sort transcripts into Bloom bins (Chapter 4).

The closest qualitative peer remains the iStar learning study [20]. It supports the first-modelling-move problem (PH1) and parts of CAT1, CAT4, and CAT7: moving from a requirements scenario to a first model is hard, and learners ask for explanation and checking rather than only a symbol list. It is also a methods peer (grounded analysis in modelling education). It is not a template that this thesis replicates. Li et al. studied one goal-modelling language after a tutorial and a modelling session; this corpus is multi-notation, interview-only, and includes lecturers.

Chakraborty and Liebel report that students often do not understand what a model “says,” and that perceptions of modelling polarise between valuable instrument and extra documentation [8]. That finding supports CAT2 as a comprehension and notation problem rather than only a missing demonstration. It does not split the start-gate (CAT1) from overload (CAT2); the present analysis treats those as distinct mechanisms.

Maslov’s BPMN education agenda [25] remains a parallel strand rather than a source of BPMN learning outcomes. It flags scaffolding, feedback, and typical novice errors as open needs. This thesis does not close that agenda; it shows how getting started, getting comments, using tools, and working in groups hang together across families.

Liebel et al. and the human-factors MDE literature [21, 22] support treating tools as socio-technical rather than as a brand choice. CAT7 versus CAT8 is that tension in interview form: semantic checks are wanted; an antique compile UI or a mid-course licence switch is still dropped. Buchmann et al. insist that models are knowledge structures, not drawings [5], which helps explain why a drawing tool that cannot catch an invalid model is a poor substitute for a checker—and why students still choose draw.io when the checker costs more than it saves.

Grassl et al. treat motivating domains and inclusion as design variables [13]; Holmes et al. describe authenticity as real projects, real tasks, and real mentors [16]. CAT10 relates to that authenticity literature and qualifies it: in these interviews a concrete reader (supervisor, stakeholders, defence) pulled care up more reliably than domain excitement alone. Modelling-specific evidence for “who reads the diagram” remains thinner than evidence for “which domain the diagram is about.”

Course load and theory–practice gaps (CAT11) have little dedicated modelling-education literature. EVELIN treats structural conditions as something competence-oriented curricula must measure [27]; that is an adjacent agenda, not a study of ECTS versus a paid job versus packed concurrent courses.

3.7.2 Additional work that strengthens selected claims

A further set of papers is useful where the core frame is thin.

Industrial modelling tools give little quality feedback, so students wait on the instructor. Hasker and Rowe built UMLint because a student may work for days, submit, and wait further days for a graded response, while producing diagrams that are syntactically acceptable and semantically wrong [14]. Foss, Urazova, and Lawrence similarly treat delayed marking of design diagrams as a teaching bottleneck and offer immediate, automated feedback on UML database-design questions [11]. Those papers support CAT5 and PH2 as a recognised educational problem: the correctness signal is late and person-bound unless a tool intercepts it. They also show that automated checkers are an available design response, not only “more lecturer hours.” They do not address CAT6. Access—who can show a half-finished diagram, in public or at the desk, on graded or ungraded work—is largely absent from that tool literature.

Silva, Gadelha, Steinmacher, and Conte compared individual and group UML exercises and found that groups were not uniformly better: some artefacts were more correct when produced individually and more complete when produced in a group [29]. That supports CAT9’s two-sided claim—groups can kill or save a diagram—and argues against treating group work as automatically the better pedagogy. Their outcome measures are correctness and completeness of classroom exercises, not code-first pressure, exclusive notation ownership, or a defence audience. Those conditions are the present study’s addition.

Sikkel and Daneva argued that textbooks rarely teach consistency across UML views and developed pairwise-comparison exercises so that students notice missing elements between diagrams [28]. Verbruggen and Snoeck observed students modelling the same case in a UML class diagram and a BPMN process model and argued that knowing which requirements belong in which perspective is part of model quality, not an extra trick [32]. Those sources support CAT14 as a recognised teaching problem: a class diagram, a process model, and a goal model answer different questions about the same system, and keeping them consistent has to be taught. They do not document the incidents in this corpus—an iStar rationale that loses its goals when the work moves to UML (S6), or a BPMN pool copied too directly into classes (S10). They also do not say whether courses teach that difference up front, or whether students only hear it after a diagram has already gone wrong.

Cámara, Troya, Burgueño, and Vallecillo assessed ChatGPT on UML class diagrams with OCL and found limited reliability: syntactic and semantic deficiencies, inconsistency, and poor scaling compared with code generation [6]. That result is consistent with CAT13’s student stance (explain and compare, do not submit generated homework): if generation is unreliable, using the tool as an explainer is a rational classroom tactic, not only an integrity rule. It also undercuts any claim that current LLMs already close CAT12 by compiling models. The present CAT13 evidence is four student cases, not an evaluation

of ChatGPT.

3.7.3 Where the literature remains thin

Taken together, prior work is strongest on landscape fragmentation, Bloom-shaped outcome gaps, iStar-specific challenges, tool friction in MDE courses, delayed diagram feedback, and multi-perspective consistency. It is weaker on feedback *access* (CAT6), groups as kill-or-save under code-first pressure, AI used as an explainer in modelling courses, and course load as a blocker of demonstrations and feedback loops (CAT11). This thesis does not close those gaps by volume of interviews; it offers a bounded, multi-notation, student-and-lecturer account against which that literature can be read.

3.8 Summary

In summary, existing work provides: (i) landscape-level evidence that modelling education research is distributed across modelling families and communities [26]; (ii) frameworks that structure modelling learning outcomes using Bloom-based dimensions [2, 3, 4]; (iii) empirical studies and agendas that identify modelling-specific learning challenges and the need for validated educational support [20, 25]; and (iv) tool-, competence-, and human-centred perspectives that emphasise the role of context, usability, motivation, inclusiveness, and authenticity [13, 16, 21, 27]. Section 3.7 places that frame against the findings of this thesis and adds a smaller set of studies on delayed diagram feedback, group versus individual UML work, multi-perspective modelling, and LLM-generated UML [6, 11, 14, 29, 28, 32]. Those additional sources strengthen selected claims; they do not replace the core literature summarised above.

METHODOLOGY

This chapter reports how the study was designed and carried out. It converts the research questions of Chapter 1 into a qualitative procedure: who was interviewed, how data were collected and analysed, how sampling stopped, and how trustworthiness and limits are bounded. The analysis follows Socio-Technical Grounded Theory (STGT) as adapted for software engineering [15, 30]. The chapter describes the procedure that was actually used—open coding, short analytical memos, constant comparison, and category development—rather than a formal Strauss–Corbin axial coding paradigm.

4.1 Research design

The study used a qualitative, theory-building design based on Grounded Theory (GT) as adapted for software engineering research [1, 30]. To attend to the interplay between social and technical elements in modelling education (students and instructors, teaching practices, notations, tools, and artefacts), the analysis was informed by STGT [15]. Data collection and analysis proceeded iteratively: wave 1 interviews seeded categories, and wave 2 recruitment was directed at open questions in that emerging map [1, 30].

The design was also informed by prior modelling-education research that combines modelling activities with interviews to elicit situated reasoning, particularly the iStar learning study [20]. That study used a tutorial and a modelling session before the interview. The present work did not replicate that activity. It used semi-structured interviews with incident probes about recent course tasks, and it targeted a broader range of modelling contexts than a single modelling method.

The intended outcome was a grounded explanatory account of modelling learning and teaching challenges and associated support needs, organised as categories and cross-cutting phenomena (Chapter 5). Design-oriented recommendations are treated as implications, not as validated intervention effects.

4.2 Research questions and operational focus

The study was guided by three research questions:

- **RQ1:** What challenges do students and instructors perceive in learning and teaching software modelling in higher education, and how do these challenges manifest across modelling activities (e.g., interpreting, creating, refining, and evaluating models)?
- **RQ2:** What strategies do students and instructors employ to address these challenges, and what support needs (pedagogical and tool-related) do they identify?
- **RQ3:** How do socio-technical factors (e.g., tools, course design, domains, feedback practices) shape modelling learning and teaching experiences?

Each research question was treated as something to explore rather than a hypothesis to test. RQ1 focused on characterising challenges and their variation across modelling activities. RQ2 focused on coping strategies and articulated support needs, including the conditions under which particular forms of support were seen as helpful. RQ3 focused on linking challenges and strategies to contextual influences. The interview guides turned these three questions into things that could be asked in a conversation. Appendix [A.3](#) lists which main prompts speak to which research question.

4.3 Study context and participants

The study was conducted in higher-education software engineering and related courses that include software modelling (for example requirements engineering, software design, business process modelling, or model-driven engineering). Two participant groups were included:

- **Students:** learners who had encountered modelling activities in at least one course and had completed at least one modelling-related assignment or project.
- **Instructors:** lecturers or teaching assistants who had taught modelling-related content and had experience assessing student models or guiding modelling activities.

Fourteen interviews were completed: four lecturers (L1–L4) and ten students (S1–S10). Table [4.1](#) summarises the corpus in general terms. Most student cases were in Portuguese higher education. Wave 2 added students at Instituto Superior Técnico (IST; S7, S10), Instituto Superior de Engenharia de Lisboa (ISEL; S8), and one further Portuguese student (S9). Wave 1 also included two Computer Science students at a university in Wrocław (S4, S5) and one Informatics master’s student in Munich (S6). Lecturers taught modelling-related courses in Portuguese higher education; they were not re-sampled in wave 2.

Table 4.1: Interview corpus (anonymised). Context is given at a general grain.

ID	Role	Wave	Context (general)
L1	Lecturer	1	Portuguese HE; process, SysML, and related modelling
L2	Lecturer	1	Portuguese HE; modelling-related teaching
L3	Lecturer	1	Portuguese HE; UML-heavy undergraduate modelling and SE
L4	Lecturer	1	Portuguese HE; requirements and modelling under load
S1	Student	1	Portuguese HE; SE course, dissertation, and work sketches
S2	Student	1	Portuguese HE; SE course, dissertation, and work sketches
S3	Student	1	Portuguese HE; BPMN / process modelling
S4	Student	1	Foreign (Poland) student; UML and ER
S5	Student	1	Foreign (Poland) student; UML team project
S6	Student	1	Foreign (Germany) student; iStar, NFR, UML
S7	Student	2	Portuguese HE; systems analysis and modelling; UML / BPMN
S8	Student	2	Portuguese HE; project-required models, then master's SE
S9	Student	2	Portuguese HE; modelling recalled at a distance
S10	Student	2	Portuguese HE; systems analysis and modelling, several notations

Inclusion emphasised direct experience with modelling tasks, so that accounts could stay close to concrete incidents. Identifying course titles, project names, and personal details are omitted or generalised here.

4.4 Data collection

Data collection used semi-structured interviews with the two stakeholder groups. Two interview guides were prepared—one per group—with a shared backbone (perceptions of modelling, challenges, strategies, and support needs) but different emphases: student interviews focused on learning experiences and task-level difficulties; instructor interviews focused on teaching goals, scaffolding, assessment, and observed misconceptions. Interviews typically lasted 20–30 minutes. Guides included incident probes that asked for one recent assignment, laboratory, or the first minutes of a text-to-model task, so that talk stayed closer to practice than to general opinion. No live modelling exercise was run in the session, and no model artefact was present for a shared walkthrough. The procedure was refined through a pilot to validate interview flow, estimate timing, and improve those probes.

Wave 2 student interviews used the same student spine, with extra probes aimed at the open questions left by wave 1 (Appendix A).

4.4.1 Interview guides and pilot

Two semi-structured interview guides were developed: one for students and one for instructors. Bloom-based modelling frameworks informed the formulation of prompts by mapping modelling competence to observable activities (interpreting models, constructing models from text, judging model quality), supporting broad coverage of cognitive demands without imposing Bloom as a coding template.

The pilot phase refined question wording, ordering, and probing strategies, with the goal of improving the specificity of examples. Pilot interviews were retained in the final dataset (S1 and S2 appear as ordinary student cases) once the protocol had stabilised; their original role was to validate the protocol, not to stand as a separate subsample.

4.4.2 Incident-based probing

To move beyond abstract opinions, the interviewer asked participants to recount a concrete episode: a recent modelling assignment or laboratory, a first-ten-minutes account of going from a text description to a model, or (for instructors) a recurring classroom moment when students struggled. These prompts are critical-incident questions inside the interview. They are not a second data-collection procedure.

The iStar learning study inspired the wish for situated talk: that work used a tutorial and a modelling session, then an interview about that shared experience [20]. The present study did not run such a session and did not do stimulated recall over a diagram brought to the interview. Uptake of the incident probes was uneven. Some participants stayed with a named course task; others remained more general (Section 4.8). The aim of the probes was still to surface decision points, typical errors, and tool- or feedback-related constraints that may not appear in opinion-only answers.

4.4.3 Interviews

Interviews were conducted individually. The full pilot interview guides are provided in Appendix A. Student interviews covered background and modelling exposure, perceived usefulness of modelling, learning difficulties, strategies for going from text to model, criteria for judging model quality, tool experiences, and motivational and contextual factors. Instructor interviews addressed teaching objectives, scaffolding, assessment design, common student misconceptions, tool choices and constraints, and perceived gaps in current pedagogy.

A summary mapping of guide questions to research questions is provided in Appendix A.3. Interviews included flexible probing to explore emerging themes, consistent with GT's iterative logic [1, 30]. Several wave 2 interviews (S7–S10) were conducted in Portuguese; analysis used English working transcripts, with Portuguese originals retained for verification of publication-level quotations.

4.4.4 Transcription and analytic identifiers

Interviews were transcribed, anonymised, and standardised to a speaker pair (*Interviewer* / *Participant*). Substantive turns were marked with stable identifiers (T001, T002, ...) so that codes and memos could point back to the transcript without inventing timestamps. Identifying details were removed or generalised. The resulting files formed the sole first-cycle analysis corpus: 14 standardised transcripts.

4.5 Data analysis

4.5.1 Grounded Theory variant and socio-technical stance

The analysis followed Grounded Theory principles as adapted for software engineering [1, 30], with a socio-technical stance aligned with STGT. Modelling education was treated as an interaction between social elements (students, instructors, feedback practices, course constraints) and technical elements (notations, tools, model artefacts, automation features) [15]. This stance informed both coding and the interpretation of categories: a difficulty was not coded as “cognition only” when the account bundled notation, tool, teaching, and assessment together.

The procedure was a lightweight STGT workflow rather than Classic, Straussian, or constructivist GT as a complete package. In particular, the study did *not* apply Strauss and Corbin’s axial coding paradigm (phenomenon, causal conditions, context, intervening conditions, strategies) as a formal stage. Categories were related later through constant comparison and a findings map (helps explain, blocks, contrasts, interacts with), not through a coding-paradigm matrix.

4.5.2 Coding and memoing

Analysis was carried out by the author. Transcripts were coded in a one-note-per-code layout: each open code received a short natural-language title, one or more turn identifiers, and a short excerpt. Codes captured modelling challenges, strategies, perceptions of value, tool interactions, and contextual constraints. Constant comparison was applied throughout: newly coded incidents were compared with previous incidents and codes to refine categories and identify variation [1].

Memos were written continuously, one idea per memo, linking codes and turns. Memos recorded analytic decisions, emerging hypotheses, and links between categories [1, 30]. Linked notes kept the chain turn → code → memo → category → phenomenon (Figure 4.1).

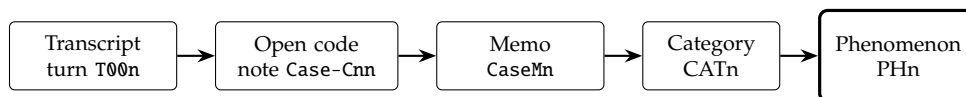


Figure 4.1: Analytic evidence chain used in this study. Identifiers are those stored in the vault notes; they are not timestamps. Selected memos and codes are reproduced in Appendix B.

The completed first-cycle corpus comprises 277 open codes and 78 analytical memos (52 student memos; 26 lecturer memos). Per-case counts are summarised in Table 4.2.

4.5.3 Category development and the findings map

After open coding and memoing, constant comparison was used to merge, split, and bound categories against the research questions. The working warrant for an overview

Table 4.2: First-cycle codes and memos by case.

Case	Codes	Memos	Case	Codes	Memos
L1	22	6	S4	15	5
L2	24	6	S5	15	5
L3	25	7	S6	15	5
L4	35	7	S7	16	6
S1	24	6	S8	9	3
S2	30	7	S9	7	3
S3	22	6	S10	18	6
Lecturer total: 106 codes, 26 memos			Student total: 171 codes, 52 memos		

category was multiple memos and multiple cases, unless a category was explicitly marked thin. Overloaded labels were split when memos distinguished different mechanisms (for example start-gate versus symbol overload; feedback timing versus feedback access; semantic checking versus tool friction).

The resulting overview comprises **14 categories** (CAT1–CAT14) and **4 cross-cutting phenomena** (PH1–PH4). Each category has a primary research-question assignment; some have a secondary assignment. Phenomena are spines that several categories feed; they are not a requirement that every category headline a phenomenon. CAT2, CAT11, and CAT14 answer research questions on the overview but do not feed a phenomenon; they condition or contrast other categories.

Lecturers’ accounts of their own teaching were kept separate from their interpretations of students. Wishes for new tools or course designs were not treated as findings.

4.5.4 Theoretical sampling

Sampling decisions were guided by emerging categories rather than predetermined demographic quotas [1, 30]. Two analytic waves were conducted (Figure 4.2).

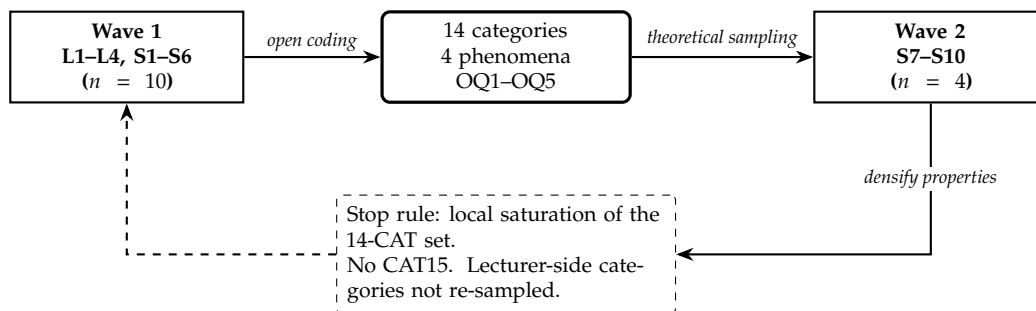


Figure 4.2: Theoretical sampling across two waves. Wave 2 densified properties of the existing 14-category set; it did not add a 15th overview category. The stop rule is local saturation of that set, not global theoretical saturation.

Wave 1. Ten interviews (L1–L4, S1–S6) were open-coded and memoed. Constant comparison produced the 14-category / 4-phenomenon overview and five open questions

Table 4.3: Cumulative overview-category count across analytic waves.

Wave	Interviews in wave	Cumulative n	Overview CATs
Wave 1 (L1–L4, S1–S6)	10	10	14
Wave 2 (S7–S10)	4	14	14

that directed the next interviews (OQ1–OQ5; Table 6.1 in Chapter 6).

Wave 2. Four further student interviews (S7–S10) were recruited as theoretical sampling against those open questions. Wave 2 added 50 first-cycle codes and 18 memos. It densified properties and bounds; it did not produce a 15th overview category that met the existing warrant. Interview density varied: S9 recalled modelling at a greater distance, so some categories rest more on the other wave 2 cases (Section 4.8).

No additional lecturers were recruited in wave 2. The student interviews were enough to densify the 14-category set for this thesis, and a second lecturer wave was not pursued. Sampling stopped when that set was locally sufficient for the research questions.

4.5.5 Local saturation of the category set

GT reporting often claims theoretical saturation when additional data no longer contribute new properties or relationships [1, 30]. That global claim is not made here. Following Dey’s caution against treating saturation as a finished theory [10], the stop rule is **local saturation of the overview category set** and **theoretical sufficiency** for this master’s research questions and sample [15, 30].

Table 4.3 reports the cumulative count of overview categories. Wave 1 produced 14 categories from 10 interviews; wave 2 left that count unchanged after four further interviews. Table 4.4 shows that “no new category” is not the same as “nothing new”: several cells record new properties or dimensions (for example CAT5 after a locked grade, CAT6 access formats, CAT14 translation), negative or boundary cases (S5, S7, S10 on the start-gate), or mere instantiation of a known property.

What is *not* claimed: global theoretical saturation across modelling courses worldwide; saturation of every property of every category; saturation of lecturer-side categories that were not re-sampled; or that unfulfilled open questions (including a lecturer “abstraction deficit” without student memos) are closed. Transferability remains analytical—conditions under which a category appears—rather than statistical generalisation [15, 30].

4.5.6 Bloom’s taxonomy as a sensitising concept

Bloom’s revised taxonomy was used as a sensitising concept to support the interpretation and communication of findings, particularly when describing the cognitive character of reported modelling challenges (understanding constructs and their semantics, applying modelling procedures, evaluating model adequacy) [18]. In practice, Bloom was applied

Table 4.4: Wave-2 densification of existing categories. Cells: empty; inst. (instantiates a known property); new (new property or dimension); neg. (negative or boundary case); — (not used: the interview did not supply an incident of that strength). S7/S10 switch incidents are counted under CAT14, not as extra CAT2 warrants.

Category	S7	S8	S9	S10
CAT1 Starting is gated	neg.	new	inst.	neg.
CAT2 Notation overload	empty	inst.	empty	empty
CAT3 Teacher examples	inst.	inst.	inst.	empty
CAT4 Self-strategies	new	inst.	inst.	inst.
CAT5 Feedback timing	new	inst.	empty	new
CAT6 Feedback access	inst.	new	new	new
CAT7 Semantic checking	inst.	inst.	empty	empty
CAT8 Tool friction	new	inst.	inst.	empty
CAT9 Groups	new	inst.	—	inst.
CAT10 Real audience	inst.	empty	empty	inst.
CAT11 Course load	new	new	empty	empty
CAT12 Good enough	new	empty	inst.	new
CAT13 AI as explainer	empty	empty	—	inst.
CAT14 Cross-notation switching	new	new	—	new

after grounded categories had been developed: codes and categories were generated inductively, and Bloom-inspired terminology was used selectively as an analytic lens and reporting vocabulary where it aligned with participant accounts. Bloom was not used as a coding scheme or set of predefined bins, consistent with grounded theory guidance for maintaining the primacy of emergent concepts [30].

4.6 Rigour and trustworthiness

Rigour was pursued following guidance for grounded theory in software engineering [30] and STGT reporting expectations [15]. This section uses the usual qualitative headings—credibility, dependability, confirmability, and transferability—rather than experimental “threats to validity” [15, 30].

Participants were not asked to review the categories after analysis (member checking). A second researcher did not independently recode the transcripts. Analysis was carried out by the author. Instead of agreement scores between two coders, a reader can follow an excerpt to a code, a memo, and a category (turn identifiers, code notes, and memos; see below).

Credibility was supported by grounding categories in experience-near accounts and by using incident probes to ask for a specific assignment, laboratory, or start-of-task moment. Those probes were verbal and uneven; they do not substitute for observation of modelling or for stimulated recall with an artefact present. Constant comparison was used to refine categories, identify variation, and retain disconfirming cases (for example students who did not wait for a teacher demonstration before drafting).

Dependability and confirmability were supported through a transparent analytic trail: turn identifiers, one-note-per-code files, memos, and an evidence map from categories to

memos and codes. Claims in Chapter 5 are written with case and memo identifiers so that excerpts can be checked. Appendix B reproduces the memos named in Table 5.3 and a small set of code notes.

Transferability is addressed by the contextual detail in Table 4.1 and by stating the conditions under which categories appeared (for example feedback access depending on public versus desk format). The thesis does not claim statistical generalisation.

4.7 Ethics and data management

Participation was voluntary and based on informed consent. Interview recordings and transcripts were anonymised, and potentially identifying details (course identifiers, project names, personal data) were removed or generalised. Data were stored securely and used only for the purposes of this research. When participants described modelling work or course artefacts, identifying details were removed or generalised, consistent with consent and institutional policies.

4.8 Methodological limitations

As a qualitative, context-sensitive study, the findings aim for analytical rather than statistical generalisation. Participant accounts may be influenced by recall bias, social desirability, and variation in course contexts. Several students (including S7 and S9) described modelling courses at a distance of years; S9 was therefore excluded from categories that required incident-level recall.

The student corpus is mostly Portuguese higher education. Two Polish cases and one German case sit in wave 1; they are not a global sample. Wave 2 added no new countries and no new lecturers. Claims that come mainly from lecturers—for example that heavy courses leave little time to teach, or that they planned several rounds of feedback but could not keep them—therefore rest on the four wave 1 lecturer interviews only. Tool- and notation-specific experiences may limit transferability across institutions or modelling families.

Incident probes reduced abstraction where a participant could name a recent task, but they do not remove recall bias: no model was shared in the session, and some accounts remained general. Sampling for variation as categories emerged, documenting contextual factors, and labelling remaining open questions rather than discarding them further bound the claims [15, 30]. What the bounds mean for particular claims is taken up in Chapter 6.

4.9 Worked example of the coding process

To illustrate the analysis procedure, this subsection shows how interview excerpts became open codes and then entered later category work. The example is drawn from two student

Table 4.5: Illustrative coding chain from two student interviews (S1, S2). Later category IDs are those used in Chapter 5.

Excerpt	Open codes (examples)	Later placement
S1 “Modelling is useful for defining how system components interact up to a certain point. After that, it can feel arbitrary and like extra documentation.”	value depends on context; modelling becomes extra documentation	Conditional value of modelling; not a dedicated overview CAT
S1 “Different arrows and boxes meant different things. . . it felt arbitrary when trying to do something useful with it.”	notation semantics unclear; symbol overload	CAT2 (notation overload); contrasts with CAT1
S1 “The tool had so many options that it became completely overwhelming to know how to make a simple entity-to-box relation.”	tool feature overload; novice usability issues	CAT8 (tool friction); related to CAT2
S2 “Some concepts may be really complex and you cannot fit them in your mind. Making a small diagram can simplify them for you and for others. . . ”	diagram as cognitive offloading; communication aid	Supports why modelling is worth the cost when an audience exists (CAT10)
S2 “I had to use frameworks with which I was unfamiliar. I had to learn them before I could start drawing.”	unfamiliar method barrier; entry cost	CAT1 / CAT2 boundary: missing path versus symbol/framework load

interviews (S1, S2). It is illustrative of the chain excerpt → open code → memo → category; it is not a claim that these two cases saturate any category.

Step 1: Open coding. Excerpts were segmented into meaning units and labelled with concise, action-oriented codes that stay close to the participant account (for example “symbol choices create doubt at start”, “tool options overwhelming for simple relations”).

Step 2: Memoing and comparison. Codes that recurred or contrasted were written up as short memos and compared with other cases. Relations such as “helps explain” or “contrasts with” were recorded when memos supported them; they were not forced into an axial paradigm.

Step 3: Category placement. Where patterns indicated a bounded mechanism, codes and memos were assigned to an overview category (and, where relevant, to a phenomenon). Support needs mentioned by participants were kept distinct from the finding itself when they were design wishes rather than observed practice.

This worked example is not a fixed coding template. It is included to show traceability from excerpts to codes and later categories. The corresponding open-code notes and

memos for these and other cited incidents appear in Appendix [B](#).

FINDINGS

This chapter reports the grounded account that emerged from the 14 interviews. It is organised by research question rather than as a catalogue of fourteen disconnected labels. The analysis produced **14 categories** (CAT1–CAT14) and **four phenomena** (PH1–PH4). Phenomena are cross-cutting claims that several categories feed; they are not a requirement that every category headline a phenomenon. CAT2, CAT11, and CAT14 answer the research questions but do not feed a phenomenon: they contrast or condition other categories.

Quotations are short excerpts from the coded transcripts, identified by case. Portuguese interviews (S7–S10) are quoted from the English working transcripts used in analysis. Claims stay bounded by memo and case identifiers; student evidence is not merged with lecturer interpretation, and design wishes are not treated as tested interventions.

5.1 How to read this chapter

Each category subsection states (i) what the category is, (ii) how it varies, including wave 2 properties, (iii) at least one short quotation, and (iv) negative or boundary cases where they exist. Table 5.1 lists all fourteen categories before the detailed sections. Figure 5.1 shows how categories sit under research questions and feed phenomena. Table 5.3 at the end of the chapter repeats the map with research questions, one-line claims, and memo identifiers (reproduced in Appendix B).

5.2 RQ1: Challenges across modelling activities

RQ1 asked what challenges students and instructors perceive, and how those challenges show up when people interpret, create, refine, and evaluate models. Four categories answer this question first: getting started (CAT1), too many symbols and frameworks (CAT2), judging whether a model is good enough (CAT12), and moving from one kind of diagram to another (CAT14). These are different problems. Waiting for a worked example

Table 5.1: Overview of the fourteen categories. Detailed evidence follows in the sections below.

ID	What it is
CAT1	Getting started is hard without a demonstration or a path already seen.
CAT2	Too many symbols, arrows, or frameworks create doubt.
CAT3	Teacher examples and walkthroughs unlock the work.
CAT4	When teaching is thin, students invent their own process.
CAT5	Feedback arrives too late for the model still to change easily.
CAT6	Who gets detailed feedback depends on asking and on public versus private format.
CAT7	Checks that catch an invalid model are valued.
CAT8	Tools add cost: options, licences, switches, weak interfaces.
CAT9	Groups can kill a diagram or keep it alive.
CAT10	A real reader makes modelling worth the care.
CAT11	Course load leaves little time for demonstrations and feedback loops.
CAT12	Without a compile result, it is hard to judge “good enough.”
CAT13	AI is used to explain or compare, not to generate graded models.
CAT14	Moving from one kind of diagram to another is a separate difficulty.

is not the same as getting lost in arrows. A class diagram can look fine as UML and still drop the goals or the process that the previous model was meant to carry.

5.2.1 CAT1 — Starting is gated without a worked path

Getting from a written problem to the first real application was often gated on a demonstration, prior contact with the same notation, or some other worked path. Lecturers described a room that waits: “They wait for the teacher to start the solution. . . Some do, but they are a minority. The others wait” (L1). Students described the same gate from the other side when they had not yet seen the notation used on a live task.

Wave 2 added a different form of the same gate. In S8’s bachelor’s project, UML and requirements models were required when S8 did not know how to use those notations, even though earlier courses had already used entity-relationship and class diagrams: “I had the idea that I had to use UML, requirements engineering, and so on, because it was in the project requirements, but I had no idea how to use those notations” (S8). A later Software Engineering course introduced modelling more explicitly, but by then a homemade sketching routine was already in place (CAT4).

The gate is not universal. S3 could start MSP exercises immediately because of earlier, even if shallow, BPMN contact: “In MSP, I remember clearly that I could start the exercises as soon as the professor showed them. I did not need to wait” (S3). S5 rarely waited for a teacher demonstration (“I rarely wait for a teacher demo”). S7 and S10 started without copying the board; wait-to-start can describe the *room* (classmates who copy) without describing every student. S9, who recalled modelling at a distance, likewise needed to see the task done once before starting. Figure 5.2 summarises those three forms of the gate.

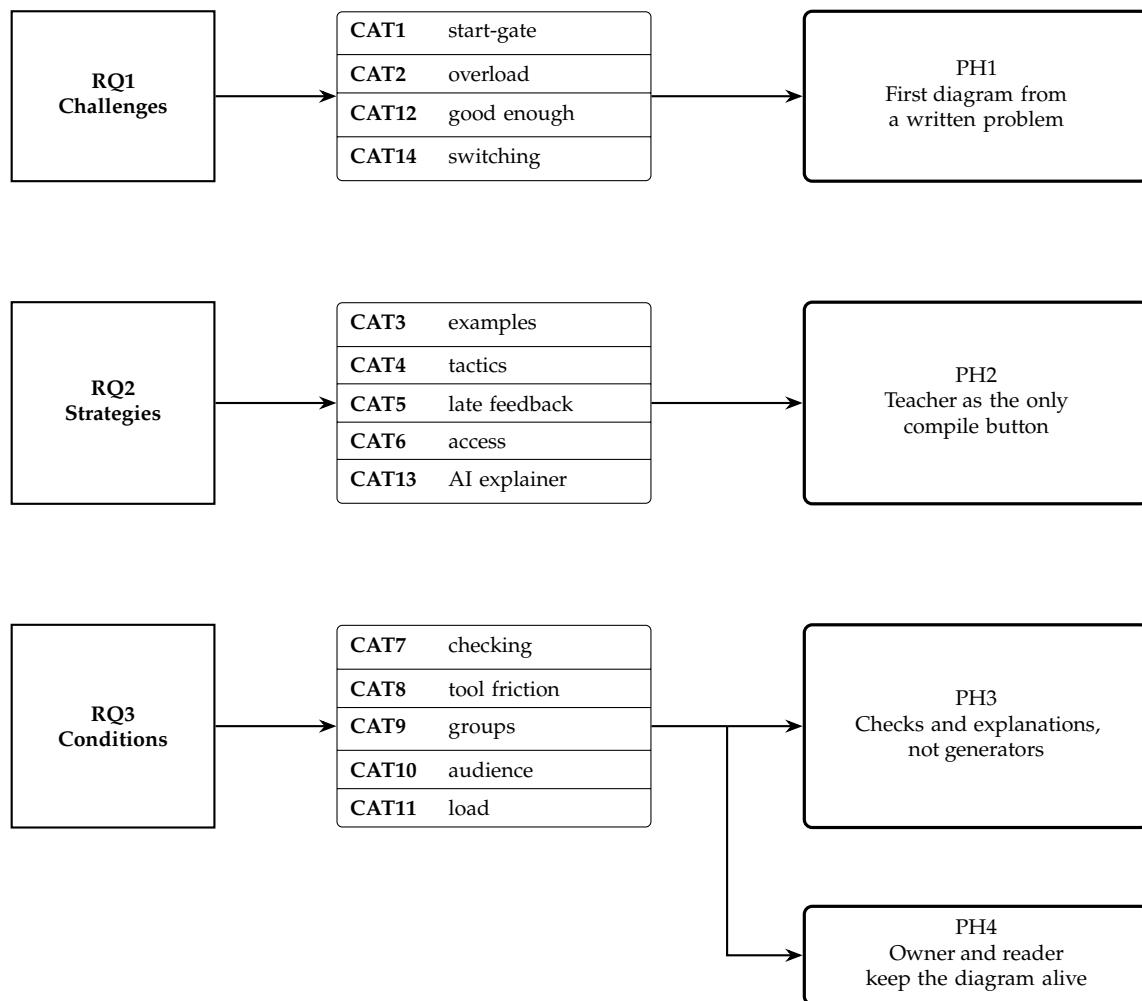


Figure 5.1: Findings map: research questions, overview categories, and phenomena. CAT2, CAT11, and CAT14 answer research questions but do not feed a phenomenon; they contrast or condition other categories (Figure 5.11).

5.2.2 CAT2 — Notation and symbol overload create doubt

Unfamiliar symbols, arrows, and frameworks created doubt at the start *and* mid-work. This is distinct from CAT1: S1 could name the high-level task and still stall on notation. “The high-level concepts were fairly straightforward, but the symbols and notation complicated things” (S1). On a game-modelling project, “Different arrows and boxes meant different things. There was a lot of information to absorb, and it felt arbitrary when trying to do something useful with it” (S1). S2 located course difficulty in having to learn frameworks before drawing: “I had to use frameworks with which I was unfamiliar. I had to learn them before I could start drawing” (S2).

Wave 2 added that similar-looking diagrams confuse until an example sits beside them (S8). When S7 and S10 got stuck moving from one kind of diagram to another, that is discussed under CAT14 rather than as more symbol overload. Figure 5.3 keeps the two mechanisms apart: a missing path is not the same as too many symbols.

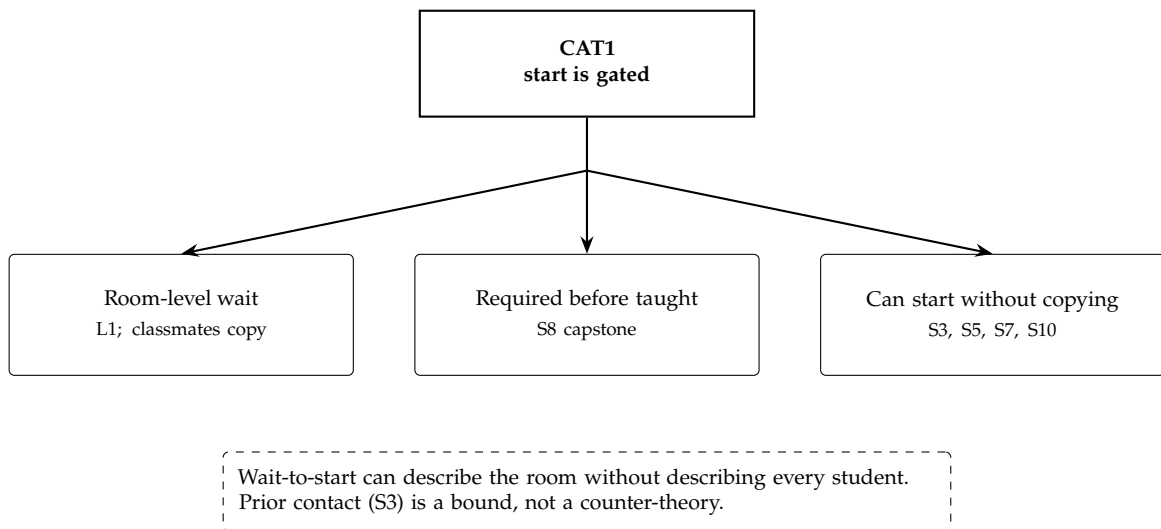


Figure 5.2: CAT1 variation: room-level waiting, a project that required notations not yet taught, and students who can start without copying (cases L1, S3, S5, S7, S8, S10).

Two mechanisms, not one “UML is hard” story

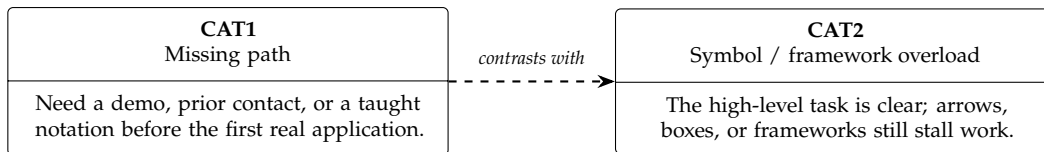


Figure 5.3: CAT1 versus CAT2. S1 could name the high-level task and still stall on notation; that incident belongs on the right, not as another start-gate.

5.2.3 CAT12 — Hard to judge good enough

Unlike code, a model does not compile. Students and lecturers described quality as walkthrough, justification, granularity, and readability, in a setting where several solutions can be valid. S4 used a homemade checklist because none was official: “I have a kind of personal checklist, although nobody gave it to us as an official rubric” (S4). S7 judged exams by a homemade list and “faith” that the teacher’s model would match. S10 added explainability for oral defence: “If three people who made the model had difficulty explaining it, it was probably too complicated” (S10).

Lecturers accepted multiple valid models (L1) and still had to grade completeness, consistency, and readability. The missing compile result is what makes CAT5 and CAT6 consequential: the teacher (or a late comment) is often the only “right/wrong” signal (PH2).

5.2.4 CAT14 — Cross-notation switching is hard

Moving across notation families lost traceability or required a new mindset and tool set. S6’s hardest moment was not drawing actors but translating an iStar rationale model into

UML: “our pair produced a class diagram that looked syntactically correct but lost the goals” (S6), and “nobody had explicitly explained how much of the iStar model should survive in UML” (S6). Each family demanded a different stance: “UML felt more structural. iStar felt organisational and political: who depends on whom, which goals conflict” (S6).

Wave 2 densified CAT14 beyond that single RE course. S10 translated an external BPMN pool too directly into UML and was told afterwards that “the models were answering different questions” (S10). Names and requirements carried; process structure did not become classes. S7 reported a lighter variant: confusion at the switch was mainly syntax and duplicated teaching, not a first start. S8 got stuck when two notations looked similar (requirements diagrams versus class diagrams). That is part similar-symbol confusion (CAT2) and part moving between kinds of diagram (CAT14). Whether lecturers teach “different questions” as syllabus content remains open (Chapter 6). Figure 5.4 is the translation, not another symbol list.

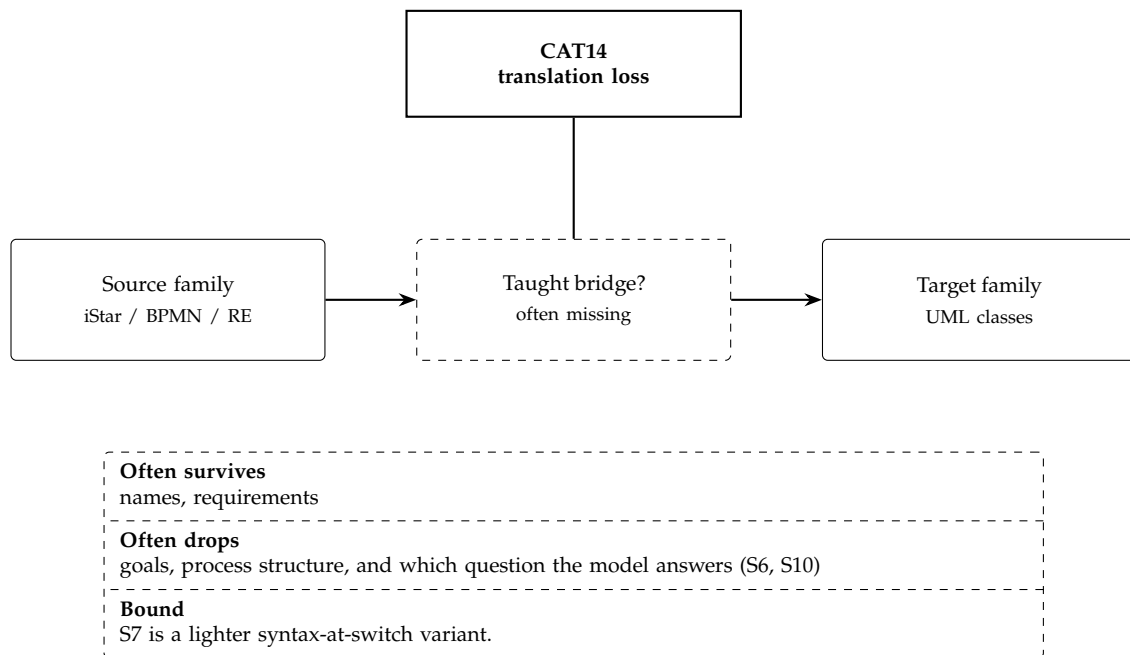


Figure 5.4: CAT14 translation loss. The missing piece is often a taught bridge: what must survive when the family changes (S6, S7, S10; not S9).

5.2.5 PH1 — Turning a written problem into the first diagram

The hard, under-taught step is making the first model from a problem text—not memorising symbol names. CAT1 (missing path), CAT3 (examples as unlock), and CAT4 (self-invented procedures when teaching is thin) jointly support this phenomenon (Figure 5.5). CAT2 can delay the same first move, but it is a different mechanism: too many symbols, not the absence of a path.

What PH1 is *not*: a claim that every student waits, or that notation learning is irrelevant.

S3, S5, S7, and S10 could start; the room can still wait. The phenomenon names the under-taught mapping from a written brief to a first diagram. Teacher examples open that mapping (CAT3); homemade tactics are what students do when the mapping is missing or late (CAT4).

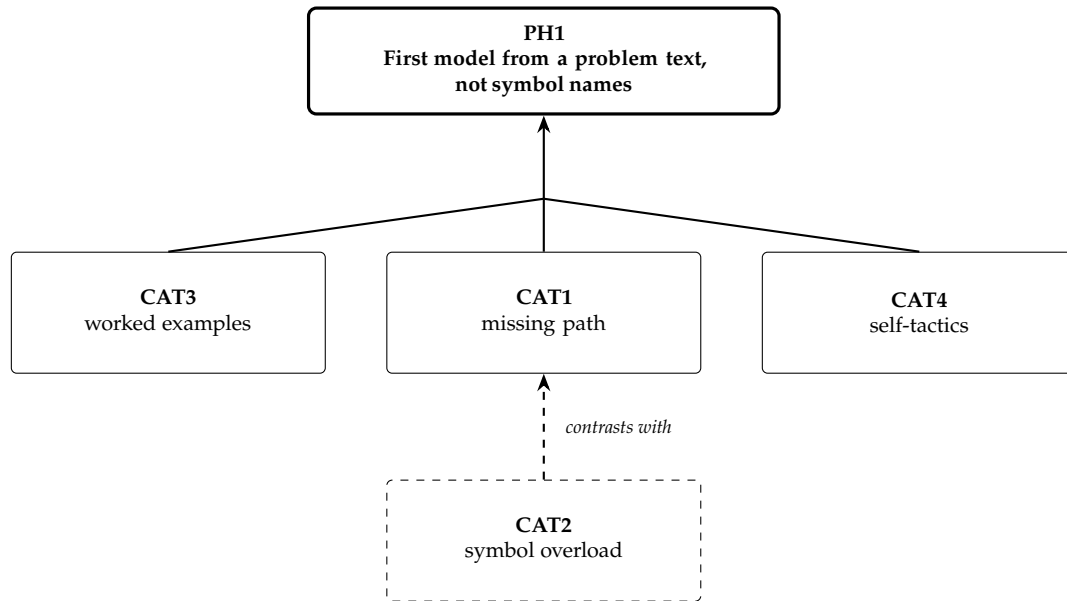


Figure 5.5: PH1 focused view. Feeding categories are CAT1, CAT3, and CAT4; CAT2 is a contrast, not a feeder.

5.3 RQ2: Strategies and support needs

RQ2 asked what people do about the challenges and what pedagogical or tool support they identify. The primary categories are teacher examples (CAT3), student-invented process tactics (CAT4), late feedback (CAT5), access to feedback (CAT6), and AI used as an explainer (CAT13).

5.3.1 CAT3 — Teacher examples and walkthroughs unlock work

Worked examples and whiteboard walkthroughs were the main pedagogical unlock. S1 put it directly: “Examples are more useful for me because I learn by seeing things” (S1). L1 described a board ritual: map the statement into a model step by step, then give another exercise to do alone. Wave 2 instantiated the same pattern: S7 recalled a simple board example followed by class debate; S8 said examples taught notation more than the expected answer.

A visible negative case is L4, who does not solve exercises up front: “No, I do not solve exercises for them. They solve the exercises with me” (L4). That practice is a pedagogical choice, not evidence that examples are unused elsewhere.

5.3.2 CAT4 — Students invent process strategies when teaching is thin

When a worked path was missing or late, students invented procedures: trial-and-error text-to-model, objects then relations, reading symbols as words, early drafts. S5's group scripted the first ten minutes before opening a tool: "Someone reads the text aloud, we each say what jumps out, and we decide who drafts the first diagram" (S5).

Wave 2 added tactics tied to classroom risk and project timing. S7 attacked the problem quickly "to have something done in case I was the 'victim' of the board" (S7), while classmates copied. S8 underlined requirements and moved from pencil to pen on the bachelor's project that required UML they did not yet know how to use; earlier courses had already used entity-relationship and class diagrams (CAT1). S10 drafted in a notebook and only then put work on the board. These are student strategies, not evidence that the course designed them. Table 5.2 lists the recurring tactics without treating them as a course method.

Table 5.2: CAT4 process tactics invented by students when a taught path was thin or late.

Tactic	Where it showed up
Read the brief aloud, then draft	S5 group script before opening a tool
Objects then relations / symbols as words	Wave-1 process talk (S1–S3)
Attack-to-survive vs wait-to-copy	S7 board-lab split
Underline, pencil, then pen	S8, before master's teaching
Notebook then board	S10, reserved starter

5.3.3 CAT5 — Feedback arrives too late

End-of-course or end-delivery feedback let errors compound. S1 wanted problems caught earlier: "Problems could have been addressed along the way instead of propagating until the end" (S1). Lecturers often planned several rounds of feedback, but class time and ECTS then cut those rounds short.

What students could still change depended on remaining time and whether the grade was already locked. S8's individual project received walk-around comments that "almost always implied changes, large or small" (S8), with no remembered full redo. S10 still had time before the next delivery and did "a large local restructuring" (S10). S7 received the grade before comments—"The grade was given before the feedback" (S7)—and a rushed teammate then forced a full rebuild: "we had to completely redo the system from the first phase" (S7). CAT9 (group tempo) interacts with CAT5 here.

5.3.4 CAT6 — Feedback access depends on initiative and format

Detailed feedback often arrived only if students asked, and public formats favoured those willing to expose a half-finished diagram. S4 preferred private channels: "Not very comfortable in public. I am shy, so presenting a half-finished diagram to the whole class is stressful" (S4). S5, more outgoing, used a public lab board. Wave 2 widened the conditions

beyond a shy/outgoing split: S7 was comfortable showing *models* (more than code) but still treated extra comments as asked-for and in person; S8 shared easily with friends and the teacher, not with course-group strangers; S9 would hide poor drawing rather than a wrong model; S10 found desk walk-around easy and raising a hand hard: “If the teacher comes by your desk and asks ‘is everything all right?’, it is much easier to show what you are doing than to raise your hand in front of the class” (S10). Graded versus ungraded work also gated whether S8 sought checks.

S10 wanted a private way to check a model during the lab. Whether that would have answered the small questions that never reached the teacher is a suggestion for teaching design, not a finding (S10). Figure 5.6 keeps timing (CAT5) separate from access (CAT6): both feed PH2, but they are not the same problem.

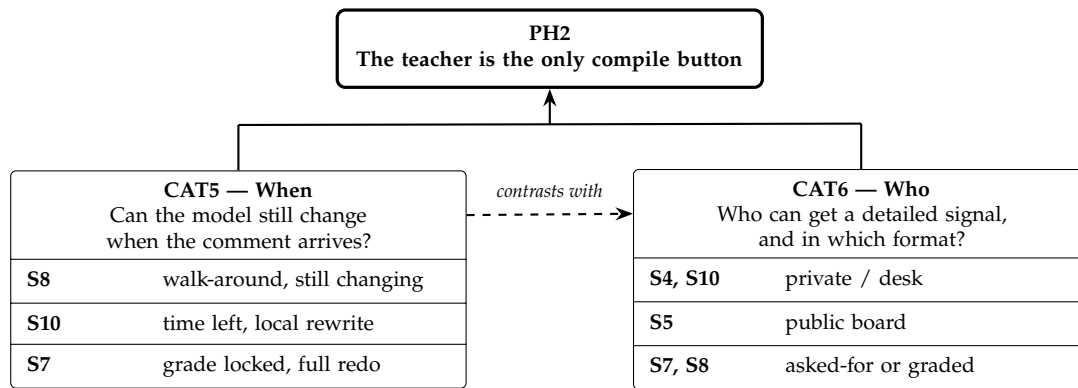


Figure 5.6: CAT5 versus CAT6. *When* comments arrive is not the same as *who* can get them (S4–S10).

5.3.5 CAT13 — AI used as explainer, not generator

Students who used AI asked it to explain options or compare approaches; they rejected generation for graded work. S5 used a chatbot “to suggest alternative class diagrams from a bullet list, then compared options with the team” (S5) and would not submit raw output. S10’s incident is the first Portuguese lived case in the corpus: screenshots and small questions after a self-check, never a full model—“I never asked ‘make me the model of this whole system’.” (S10). That use also substituted for some public questions S10 avoided (CAT6). S4 and S6 reported the same explain-not-generate stance, which later feeds PH3 (Section 5.4.3). One lecturer wanted an LLM that would explain a student’s model back in ordinary language, so the student could compare that explanation with what they had meant—not a tool that draws the assignment (L3). That is a teaching idea, not a CAT13 finding; Chapter 6 returns to it.

5.3.6 PH2 — The teacher is the only compile button

Because a model does not compile (CAT12), students often learn they are wrong when a teacher—or late feedback—tells them (CAT5, CAT6). Walk-around, private messages,

and public boards change *who* gets that signal in time, not the fact that the signal is social (Figure 5.7). Homemade checklists and “faith” that the teacher’s model will match (S4, S7) are student work-arounds for the missing compile result, not evidence that courses already supply one.

What PH2 is *not*: a claim that automated checkers do not exist, or that lecturers refuse to give comments. Checkers appear under PH3 (Section 5.4.3); here the point is that, in these courses, the everyday right/wrong signal is a person.

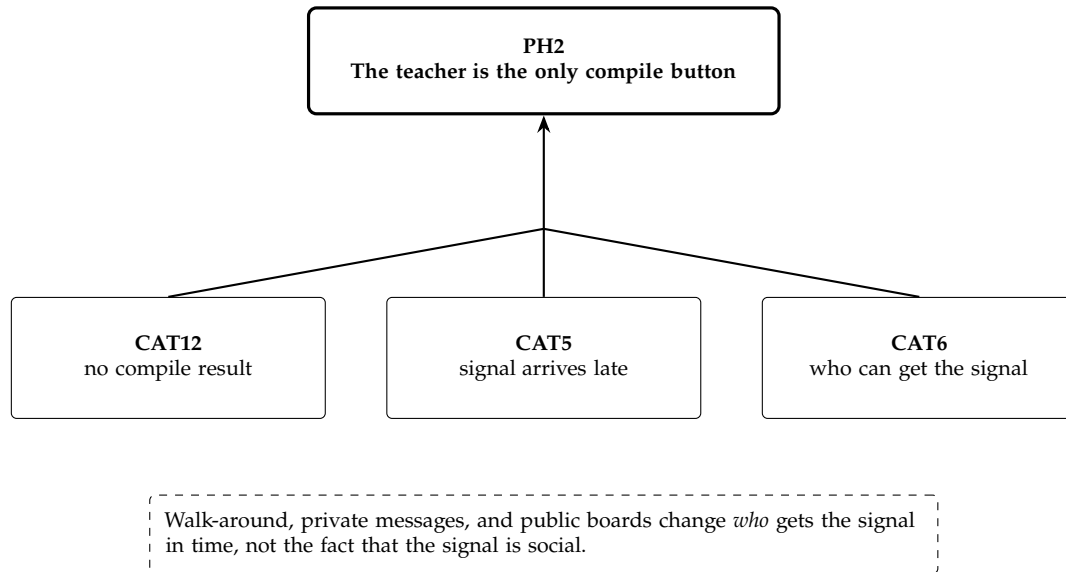


Figure 5.7: PH2 focused view. CAT12 names the missing compile result; CAT5 and CAT6 name when and how the social substitute arrives.

5.4 RQ3: Socio-technical conditions

RQ3 asked how tools, course design, domains, and feedback practices shape modelling experience. The categories here are semantic checking (CAT7), tool friction (CAT8), groups (CAT9), real audiences (CAT10), and course load (CAT11). PH3 gathers the tool and AI support claim after CAT7 and CAT8; CAT13, reported under RQ2, feeds it as well. Course load cuts time for demonstrations and feedback.

5.4.1 CAT7 — Semantic checking is valued

Tools and checks that catch invalid models were wanted. This is not a claim that heavy tools always beat drawing. S3 valued a BPMN-specific interface: “it presents a lot of the vocabulary directly in the interface” (S3). Lecturers valued checkers such as piStar for iStar (L4) and preferred modelling tools over generic drawing when licensing allowed (L1).

Wave 2 added bounds. S7 saw an open-source compile tool presented and then dropped it: “the interface was a bit antiquated and using it created more problems than it solved” (S7). S8 skipped optional Camunda/Archi suites and used draw.io. Wanting

checks (CAT7) and abandoning a checker for UI or optionality are compatible; they are not a new category.

5.4.2 CAT8 — Tool friction, switching, and cost

Option overload, licensing, mid-course switches, and slow or corrupt tools raised the cost of modelling. S1 found StarUML overwhelming for a simple relation: “The tool had so many options that it became completely overwhelming to know how to make a simple entity-to-box relation” (S1) and preferred a simpler drawing tool. Lecturers reported licensing pushing tool changes (L1). Wave 2 added an outdated compile interface that S7 then dropped, optional Camunda and Archi that S8 skipped in favour of draw.io, and UML menus S9 found outdated. Checks are still wanted (CAT7); a tool that is too complicated, optional, or slow will still be dropped.

5.4.3 PH3 — Want checks and explanations, not auto-drawn homework

PH2 is the everyday situation: the right/wrong signal is a person. PH3 is what participants asked for instead: validation and explanations, not a machine that draws the assignment (Figure 5.8). CAT7 (semantic checking valued), CAT8 (friction when tools are heavy), and CAT13 (AI as explainer) feed this phenomenon. CAT13 was reported under RQ2; it belongs here as a feeder, not as a second home for that category.

What PH3 is *not*: evidence that a particular checker or chatbot works, or that lecturers’ LLM redesign wishes (L3) are findings. Those wishes are implications (Chapter 6). CAT13 rests on four student cases, including one Portuguese incident (S10); it should not be over-read as an AI-education evaluation.

5.4.4 CAT9 — Groups can kill or save the model

Code-first pressure or passive teammates killed diagrams; ownership, pairs, live boards, and stand-ups kept them alive. S4’s group wanted to jump to implementation on a generic shop domain while S4 was still drawing the class diagram. Wave 2: a teammate who rushed to finish phase 1 forced S7’s pair into a from-scratch redo (CAT5 interaction). S10’s group split the work by notation—one person did BPMN, another requirements, another UML—and the diagrams started to disagree: “Then we had two versions of the same system” (S10). They later added a draft-and-review step so everyone could still explain the models at the defence. Figure 5.9 is that kill-or-save split; S9 is not used here.

5.4.5 CAT10 — Real audience makes modelling worth it

A concrete reader—supervisor, stakeholders, teaching assistant—pulled quality up more than domain excitement alone. S4 still held that “Interesting domains make a big difference” (S4), but wave 2 placed audience above that: course-only models “would probably only pass through the teacher’s hands” (S7), whereas a thesis architecture for supervisor and

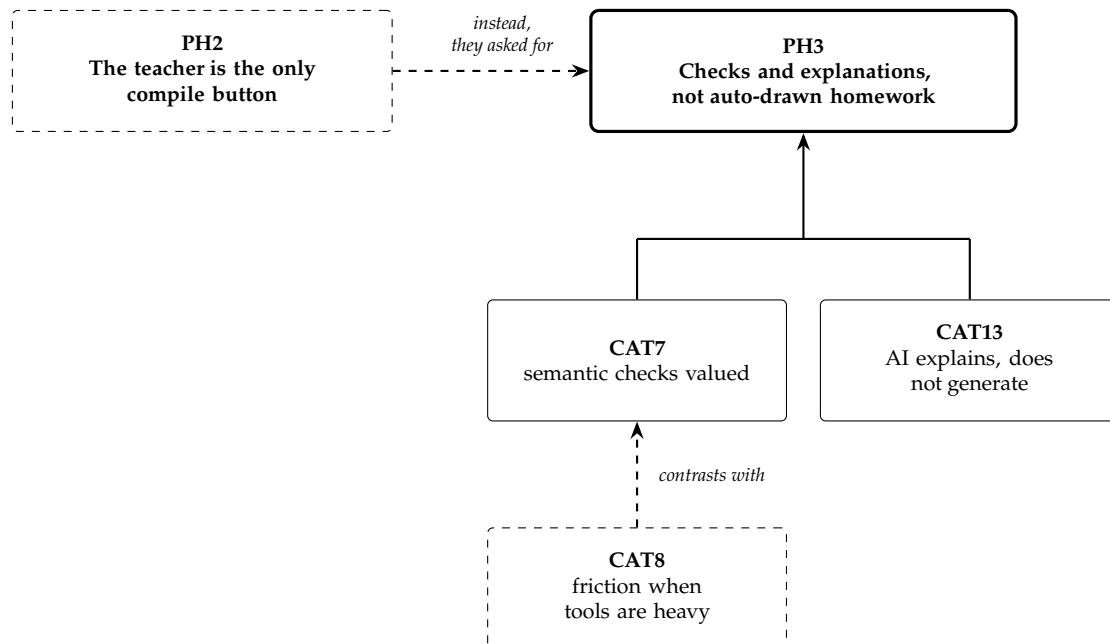


Figure 5.8: PH3 focused view. PH2 is the person-signal they actually get; PH3 is the check and explainer they asked for. Checking is wanted (CAT7); a heavy, optional, or antique tool is still dropped (CAT8); AI is used to explain options, not to generate graded models (CAT13).

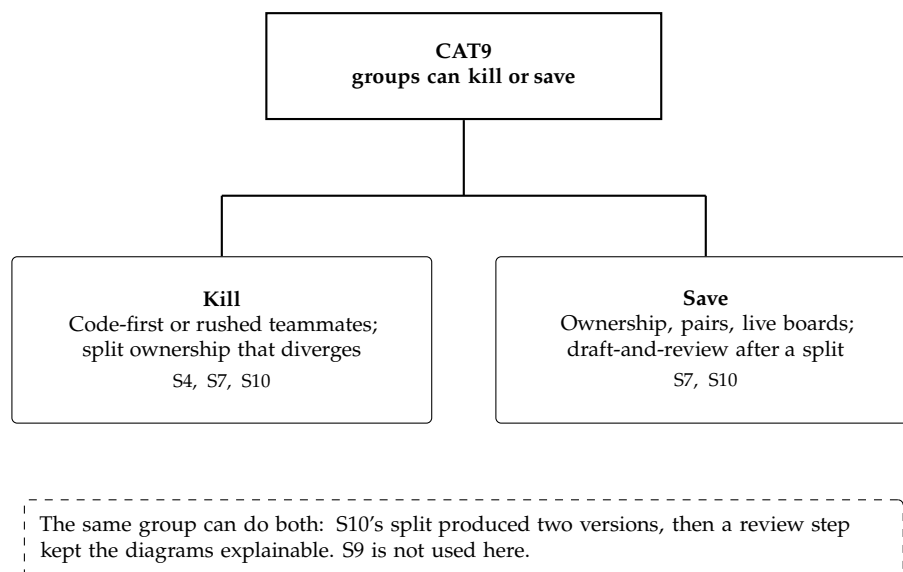


Figure 5.9: CAT9: groups can kill a diagram or keep it alive (S4, S7, S10; not S9).

defence became a priority. S10 similarly treated explainability as a defence criterion (CAT12). Domain interest matters; it does not replace a reader.

5.4.6 CAT11 — Course load and theory–practice gaps

Heavy continuous assessment, missed theory, slide-only catch-up, and late recognition of modelling’s value shaped what teaching could actually deliver. Lecturers described workload and continuous evaluation cutting attendance and time for demos and feedback (L1, L4). Wave 2 added a student dimension that is not the same as lecturer ECTS-scarcity: S7 tried to finish modelling quickly because a heavy semester left little interest—“because modelling was not something that interested me much, I tried to get those course tasks done as fast as possible so I could focus more on the others” (S7). S8’s paid tutoring job lowered interest more than other courses. When load is this high, there is less time for the demonstrations (CAT3) and the feedback rounds (CAT5) described earlier.

5.4.7 PH4 — A diagram survives only if someone owns it and someone reads it

Group ownership (CAT9) and a real audience (CAT10) jointly decide whether modelling stays alive and careful (Figure 5.10). A well-taught example (CAT3) does not keep a diagram alive in a group that has already switched to code. Course load (CAT11) is drawn as a blocker of the loops, not as a fourth feeder: when ECTS and concurrent work leave no time for demos or comments, ownership and audience have less to work with.

What PH4 is *not*: a claim that interesting domains do not matter. S4 still held that they do. Domain excitement is a weaker pull than a supervisor, a defence panel, or another stakeholder who will actually read the diagram.

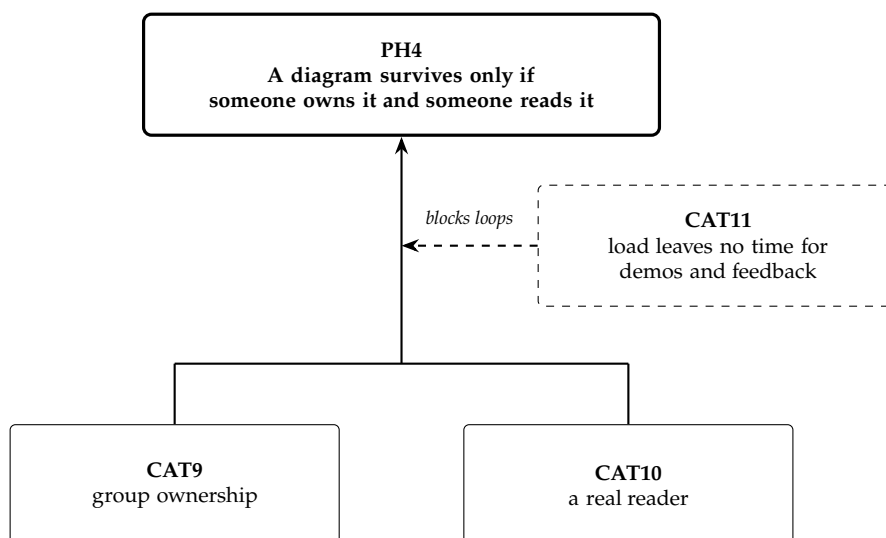


Figure 5.10: PH4 focused view. CAT9 and CAT10 feed the phenomenon; CAT11 blocks the teaching loops around it.

5.5 Category summary

Table 5.3 lists each overview category with its primary research question, a one-line claim, and the memos that most strongly support it. Those identifiers point to the analytical notes reproduced in Appendix B; they are not extra quotations. Figure 5.11 shows every CAT–CAT edge from the findings-map relations canvas, split by destination so arrows do not share one network. How categories feed PH1–PH4 is in Figures 5.5–5.10. This is not a finished global theory.

The table is the findings of this chapter. Teaching ideas and open questions that did not earn a category are taken up in Chapter 6 (Table 6.1), not as extra rows here.

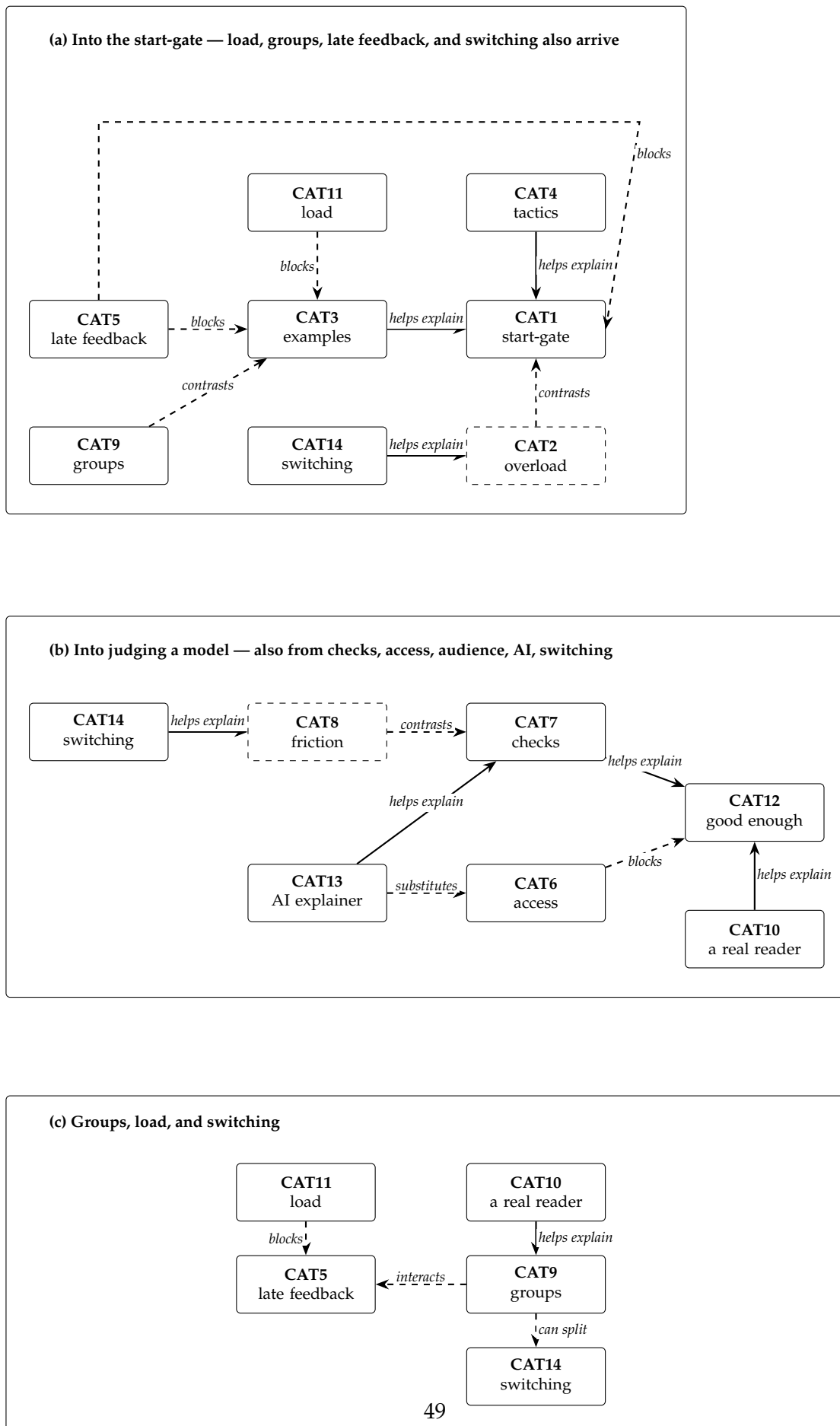


Figure 5.11: All category-to-category relations from the findings map. Solid arrows: helps explain. Dashed: contrasts, blocks, interacts, substitutes, or can split. Panels are grouped by where the arrows arrive, not by a single phenomenon: load, groups, late feedback, and

Table 5.3: Overview categories, primary research question, one-line claim, and strongest memos.

ID	RQ	Claim	Memos
CAT1	RQ1	First application is gated on a demo, prior contact, or a project that required notations not yet taught.	S3M1, S8M1, L1M1
CAT2	RQ1	Symbol and framework overload create doubt; distinct from a missing path.	S1M1, S3M3, S2M3
CAT12	RQ1	No compile result; quality is walkthrough, justification, granularity.	S6M2, S4M2, S10M5
CAT14	RQ1	Switching families loses trace or requires a new question, not only new symbols.	S6M1, S10M1, S7M5
CAT3	RQ2	Teacher examples and walkthroughs unlock work (L4 withholds solved examples).	S1M2, S7M1, L4M6
CAT4	RQ2	Students invent process tactics when teaching is thin.	S5M3, S7M1, S8M1
CAT5	RQ2	Late feedback lets errors compound; what can still change depends on time and locked grades.	S1M5, S7M3, S10M3
CAT6	RQ2	Access depends on initiative, graded work, and public versus desk format.	S4M3, S8M2, S10M2
CAT13	RQ2	AI is used to explain or compare, not to generate graded models.	S4M5, S5M4, S10M4
CAT7	RQ3	Semantic checking is valued; a heavy or optional checker may still be dropped.	S3M4, S7M2, L4M5
CAT8	RQ3	Licensing, option load, switching, and weak UI raise modelling cost.	S1M4, S7M2, L1M5
CAT9	RQ3	Groups kill diagrams under code-first pressure or save them via ownership.	S4M1, S7M3, S10M5
CAT10	RQ3	A concrete reader pulls quality up more than domain excitement alone.	S7M5, S10M5, L4M3
CAT11	RQ3	Load and theory–practice gaps block demos and feedback cycles.	L1M3, S7M6, S8M3

DISCUSSION

This chapter reads the 14-category, four-phenomenon account against the research questions and against related work. Claims stay those already supported in Chapter 5. Implications are design-oriented readings of the data, not evaluated interventions. Remaining open questions are kept as gaps, not closed by rhetoric.

6.1 Answering the research questions

6.1.1 RQ1 — Challenges across modelling activities

The challenges that earned overview categories are not a single “students struggle with UML” story. Starting is gated when a worked path is missing or arrives after a project already required models (CAT1). That gate is distinct from symbol and framework overload (CAT2). Evaluating a model is hard because there is no compile result (CAT12). Switching notation families is a further challenge: locally sensible diagrams lose trace or answer the wrong question (CAT14).

This pattern is consistent with published modelling-education work that reports difficulty moving from requirements to models and judging adequacy [20, 26, 8], but it is not a replication of an iStar-only error taxonomy. Li et al. studied novices in one goal-modelling language and derived teaching and tool requirements from that setting [20]. The present corpus spans BPMN, UML, iStar, ER, and courses that teach several of those notations together, and it includes lecturers. The closest empirical overlap is the first-modelling-move problem (PH1) and the demand for support that explains constructs rather than only listing them. Divergence appears where this study splits start-gate from symbol overload, and where CAT14 treats family translation as its own challenge rather than as more notation learning.

CaMeLOT and related Bloom-informed frameworks argue that modelling education under-specifies evaluation and metacognitive outcomes [2, 3, 4]. CAT12 and PH2 give an empirical reading of that gap: students invent checklists and wait for the teacher because the course rarely supplies a compile-like signal. Those frameworks are tools for writing course learning outcomes. This study did not sort interviews into Bloom levels (Chapter 4).

6.1.2 RQ2 — Strategies and support needs

The dominant pedagogical unlock is still the worked example and the walkthrough (CAT3), with a visible negative case where a lecturer withholds solved exercises (L4). When that unlock is thin or late, students invent process tactics (CAT4): reading the brief aloud, attacking the board to survive a random pick, pencil-to-pen sketches, notebook-then-board. Feedback is both a timing problem (CAT5) and an access problem (CAT6). AI, where it appears, is used as an explainer and a second opinion, not as a generator of graded models (CAT13).

Support needs that participants articulated therefore cluster around (i) seeing a mapping from text to model, (ii) getting a correctness signal while the artefact can still change, and (iii) getting that signal in a format they will actually use (desk walk-around versus raising a hand; private versus public). Tables 6.2 and 6.3 summarise the wave 2 variation behind (ii) and (iii). Research on teaching with modelling tools similarly argues for feedback written for learners, not the cryptic error messages of an industrial compiler [21]. Delayed instructor critique of student diagrams is itself a documented bottleneck, which automated checkers try to shorten [14, 11]. This study adds the access dimension: the same walk-around that helps a reserved student is not the same as a public board that an outgoing student enjoys, and neither is the same as comments that arrive after the grade is locked.

CAT13 should not be over-read as an AI-education finding. Four student cases, including one Portuguese incident, reject generation for graded work. That aligns with PH3 (checks and explanations, not auto-drawn homework) and with assessments of ChatGPT on UML that report limited reliability compared with code generation [6]. It does not evaluate any particular tool. L3's wish for an LLM that explains a student's model back, rather than writing it, is an implication (Section 6.2), not part of CAT13.

6.1.3 RQ3 — Socio-technical conditions

Tools, groups, audiences, and course load co-shape whether modelling happens at all. Semantic checking is valued (CAT7) and still dropped when the interface costs more than it saves (CAT8, S7). Groups can kill or save the diagram (CAT9). A real reader pulls care up more reliably than domain excitement alone (CAT10). Load and theory–practice gaps block the very demos and feedback loops that CAT3 and CAT5 require (CAT11).

Liebel et al. already showed that tool perception depends on usability, the course role of models, and support—not brand alone—and that industrial tools can demotivate when models must generate code precisely and help is unreliable [21]. CAT7 versus CAT8 is the same tension in interview form: participants want checks; they will not pay an antique compile UI or a mid-course licence switch to get them. Buchmann et al. argue that models are knowledge structures, not drawings [5]. CAT14's "models answer different questions" is a student-language version of that claim at family boundaries, consistent with teaching materials that treat cross-diagram consistency and UML–BPMN perspectives as learnable

content rather than as extra symbols [28, 32]; the missing piece in teaching is often the bridge method, not another symbol list.

Classroom comparisons of group versus individual UML exercises find that groups are not uniformly better on correctness and completeness [29]. CAT9 is the same two-sided result under different conditions: code-first pressure and exclusive notation ownership can kill a shared diagram; draft-and-review can save it.

Survey work on domain choice finds that students prefer personally meaningful domains and that educator assumptions about games and technical domains can miss that preference [13]. CAT10 does not deny domain interest (S4). It qualifies it: without a concrete reader, even an interesting domain is a weak pull compared with a supervisor or defence audience. Authenticity here is social (who will read this) as much as topical (what the system is about).

Maslov’s bibliometric picture of conceptual modelling education as fragmented across notations [26] matches the practical cost of CAT14. An agenda for empirically validated BPMN pedagogy [25] is complementary: this thesis does not offer BPMN learning outcomes; it shows how getting started, getting feedback, using tools, and working in groups hang together across different modelling courses.

6.2 Implications

The following implications are *not* findings and were not trialled.

Feedback while the grade is still open. Staged comments after a locked grade produce full redos or unused advice (CAT5, S7). Walk-around on an individual project can still change the artefact (S8) when time remains (S10). If ECTS leaves room for only one modelling submission, comments while the teacher walks the lab may be the only chance to change the work (CAT11).

Format of the correctness signal. Public boards help some students and freeze others (CAT6). Desk-side questions recovered work that raising a hand did not (S10). A private checker is a plausible recovery for small unresolved doubts; it is a design hypothesis (S10), not evidence that such a checker works.

Checks, not generators. PH3 and CAT13 point in the same direction for both modelling tools and AI: validation, explanations, and tools that only offer allowed symbols are closer to what participants asked for than auto-drawn homework. L3’s proposal belongs here rather than in CAT13: an LLM that explains the student’s model back in ordinary language, not one that writes the diagram or the constraints. That reading is consistent with education-tailored feedback arguments in MDSE teaching [21] and should not be confused with a product requirement specification.

Teach the bridge between families. Where several notations are assessed, students need an explicit method for what must survive a translation (CAT14, S6, S10). Reusing one case study across families is a teaching wish in the corpus, not a tested curriculum.

Keep a reader in the loop. Ownership inside the group and a real audience outside it (PH4) are conditions under which diagrams stay alive. Peer review after split-by-notation can surface divergence; it does not by itself create a shared model (S10).

6.3 Limits of the claims

Chapter 4 stated how quality was pursued. This section states what those bounds do to the argument.

Student-side CAT1 and CAT4–CAT6, CAT9, and CAT12–CAT14 are denser after wave 2 than at the wave 1 freeze. No further lecturers were interviewed, so accounts of course load and of planned feedback that could not be kept still come from wave 1. CAT11 therefore remains partly the lecturers’ view of institutional constraint, even though S7 and S8 added student load.

Wave 1 left five sampling questions (OQ1–OQ5). They are gaps, not extra categories. No lecturer and student from the same course offering were interviewed, which is why OQ3 is only partly answered: lecturers’ “students wait to start” can describe the room without describing every student.

The stop rule is local saturation of the 14-category set, not global theoretical saturation. Further Portuguese student interviews of the same kind would be unlikely to earn CAT15 on the present warrant; they would also be unlikely to saturate lecturer-side categories or non-Portuguese programmes.

Smaller leftover questions from individual memos (whether a checker is dropped only when it is optional; whether lab walk-around can replace a second submission; whether exclusive notation ownership was the teacher’s design or a group habit) stay as future-work recipes, not as a recruitment plan for this thesis.

Table 6.1 lists the five sampling questions, what wave 2 did with them, and what remains.

Table 6.1: Open questions that directed wave 2.

ID	Question	Status	What remains
OQ1	Is never reworking a model its own challenge?	Partly answered	Variation sits in CAT5 (when comments arrive relative to a locked grade), not a new category. How common a full redo is still open.
OQ2	Do shy and outgoing students need different public-feedback designs?	Answered	CAT6 is wider than shy versus outgoing. A private lab check remains a teaching idea, not a finding.
OQ3	Do lecturers' "students wait to start" claims match what students say?	Partly answered	Can be true of the room, not of every student. No matched lecturer–student pair in the same course.
OQ4	Is switching between notations more than one RE case?	Partly answered	CAT14 now includes S6, S7, and S10. Whether lecturers teach "different questions" as syllabus content is still open.
OQ5	Do students describe an abstraction deficit the way lecturers do?	Not answered	No student volunteered this. Wave 2 did not ask students whether they lacked abstraction, so it stays a lecturer interpretation.

The wave 2 densification of CAT5 and CAT6 is easier to scan as two compact tables than as a second reading of Chapter 5. Table 6.2 records three rework modes on the same timing-and-stakes mechanism. Table 6.3 records access conditions that are wider than a shy-versus-outgoing split. Neither table introduces a new category.

Table 6.2: CAT5 rework modes after comments (wave 2). Variation is a dimension of late feedback, not a dedicated refine category.

Case	Rework mode	Condition
S8	Local changes (no remembered full redo)	Walk-around comments on a still-open individual project
S10	Large local rewrite	Time remaining before the next staged delivery
S7	Full redo of the next phase	Grade already locked; rushed teammate froze the first-phase model

Table 6.3: CAT6 access conditions. Extra comments still tend to be asked-for; the format of the channel varies.

Case	Access pattern
S4	Shy; prefers a private teacher channel over a public half-finished diagram
S5	Outgoing; uses a public lab / group board
S7	Comfortable showing models (more than code); extra comments still in person and asked-for
S8	Easy with friends and the teacher, not with course-group strangers; ungraded work made checking optional
S9	Would hide poor drawing rather than a wrong model (thin recall)
S10	Reserved starter; desk walk-around easy, raising a hand is not

6.4 Future work

A later study can take up the remaining questions without pretending that this master's project can still collect that sample. Table 6.1 already labels what wave 2 closed, what it only partly answered, and what it left untouched. The items below are recipes for a different project, not a recruitment plan for this thesis.

- Observe the first minutes of a board laboratory, or interview the lecturer of the same offering as the students, to test wait-to-start as a room-level versus person-level claim (OQ3 remainder).
- Interview a lecturer who teaches multi-notation analysis or requirements engineering specifically about whether a bridge method—what must survive a family translation—is in the syllabus (OQ4 remainder).
- Collect non-leading incidents about choosing granularity and what to omit, or observe the same task, before treating an “abstraction deficit” as a student finding (OQ5).
- Trial a private in-lab validator with reserved students if the research question is about recovering the small doubts that never reach a public board (CAT6 remainder)—as a design study, not as an extension of these interviews.
- Compare other groups in the same systems-analysis setting on whether a full redo is common or was tied to one *despachado* teammate (S7), and whether exclusive notation ownership was the teacher's design or a group habit (S10).

Global saturation across countries, notations, and lecturer practice remains out of reach for a master's timeline. That is a different project. The local sufficiency of the fourteen-category set for the present research questions is argued in Chapter 4; it is not a claim that those later studies are unnecessary.

CONCLUSION

This thesis asked how software modelling is experienced, taught, and learned in higher education, treating modelling as a socio-technical activity rather than as notation drill alone. Fourteen interviews (four lecturers, ten students) were analysed with a lightweight STGT procedure: open coding, short memos, constant comparison, and a findings map. The result is an empirically grounded account of **14 categories** and **four phenomena**, sufficient for the research questions of this MSc, not a finished global theory.

7.1 Answers to the research questions

On RQ1, the main challenges are a gated first modelling move, symbol and framework overload as a distinct problem, difficulty judging “good enough” without a compile result, and breakdowns when students switch notation families. The four phenomena in Chapter 5 do not replace those categories: they name the cross-cutting stories that several categories feed (Figures 5.1 and 5.11).

On RQ2, teacher examples remain the principal unlock, students invent process tactics when teaching is thin, feedback is both late and unequally accessible, and AI—where used—explains and compares rather than generating graded models.

On RQ3, semantic checks are wanted but tool friction is costly, groups and real audiences decide whether diagrams survive, and course load blocks the demonstrations and loops that the other categories require.

7.2 Contributions

The contribution is a bounded, traceable map of modelling-education challenges and support needs that connects student and lecturer voices to the socio-technical conditions of courses, tools, groups, and feedback. Relative to method-specific error taxonomies and Bloom-based learning-outcome frameworks, the account splits mechanisms that are often collapsed (start-gate versus symbol overload; feedback timing versus access; checking wanted versus tools dropped) and treats family translation as its own challenge.

Implications for teaching and tools follow the phenomena: keep a worked path into the first diagram; supply a correctness signal while the artefact can still change and in a format students will use; prefer checks and explanations over generators; teach bridges between families; keep an owner and a reader in the loop. Those implications were not trialled here.

7.3 Limits

Sampling stopped when wave 2 densified properties without adding a 15th overview category, and when further interviews were no longer realistic. That is local saturation of the category set plus a recruitment constraint—not global theoretical saturation. Lecturer-side categories were not re-sampled. Several open questions remain labelled for later work (Table 6.1). Transferability is analytical: the conditions under which a category appeared, not a statistical claim about modelling courses in general.

7.4 Closing

Future work can take up the unfulfilled questions—matched course pairs, syllabus-level translation teaching, non-leading tests of abstraction, and design trials of private checkers—without discarding them or claiming they were closed. The recipes are in Section 6.4; they are a different project from the map this thesis has already earned.

PILOT INTERVIEW GUIDES

This appendix provides the semi-structured interview guides used in the pilot phase and retained, with flexible probing, for the main corpus. The guides are designed to elicit: (i) perceived challenges across modelling activities (RQ1), (ii) coping strategies and articulated support needs (RQ2), and (iii) socio-technical shaping factors (RQ3). Questions are asked flexibly; probes are used selectively depending on time and participant responses.

Wave 2 student interviews (S7–S10) used the same student spine. Extra probes on start-gate, rework, feedback access, groups, cross-notation switching, and course load were used where they spoke to open questions from wave 1. They are a sampling aid, not a second protocol.

A.1 Instructor pilot interview guide (20–30 min)

0) Opening (30–45s)

Script (say verbatim).

Thanks for meeting with me. I’m studying how modelling is taught and learned in higher education, focusing on challenges, coping strategies/support needs, and how context (tools, course design, domains, feedback) shapes the experience. With your permission, I’d like to record so I don’t miss details. The interview should take about 20–30 minutes. You can skip any question.

1) Teaching context (3–4 min)

Q1. Teaching background and where modelling fits.

- *Main prompt:* Could you briefly describe your teaching context and where modelling fits into your courses?
- *Probes (optional):* courses/level; modelling families (UML/BPMN/ER/iStar/MDE); years taught; class size; team vs individual work.

2) Learning goals and course design (6–7 min)

Q2. Intended modelling competence.

- *Main prompt:* When a student finishes your course, what should they be able to do with modelling?
- *Probes:* key activities (interpret/create/refine/evaluate); minimum vs excellent; what “good enough” looks like.

Q3. Scaffolding from problem statement to model (and better model).

- *Main prompt:* How do you scaffold students from the problem statement to a model and a better model?
- *Probes:* explicit step-by-step approach; where students get stuck; quality criteria taught explicitly vs via feedback; reflection/comparison tasks.

Q4. Assessment focus across modelling activities.

- *Main prompt:* How do you assess modelling competence in your course?
- *Probes:* final artefact vs process/reasoning; which activities assessed; quality dimensions graded (completeness/consistency/readability/fit-for-purpose).

3) Observed student challenges (6–8 min)

Q5. Biggest recurring challenges.

- *Main prompt:* What are the most common challenges students face in modelling?
- *Probes:* conceptual (semantics) vs procedural (how to build/refine); which activity is hardest; differences by notation; beginner vs advanced differences.

Q6. Concrete incident walkthrough (high value).

- *Main prompt:* Can you recall a specific moment when students struggled with modelling? What was happening?
- *Probes:* what exactly stuck; how you noticed; conditions (tool, time pressure, group work, unclear requirements, domain unfamiliarity); redesign ideas.

Optional mini-elicitation (2–3 min)

Prompt.

- Could you briefly describe one modelling assignment/exam task you use, and what a strong vs weak submission looks like?
- *Probes:* which activities it tests; diagnostic mistakes; what feedback/support would prevent them.

4) Coping strategies and support needs (6–7 min)

Q7. What you do when students struggle.

- *Main prompt:* When students struggle, what do you typically do to help, and what seems to work best?
- *Probes:* worked examples/templates/checklists/peer review; tailoring by difficulty type; what students do effectively/ineffectively.

Q8. Tool-related supports and constraints.

- *Main prompt:* What role do tools play in these challenges, and what tool support would be most helpful?
- *Probes:* syntax checking, hints, layout/views, versioning/collab; tool friction (setup/usability/cognitive load); tool changes due to student experience.

Q9. Feedback and iteration.

- *Main prompt:* How do you provide feedback on models, and how much opportunity do students have to revise?
- *Probes:* one-shot vs iterative; explicit quality criteria; approaches that improved students' evaluate/refine ability.

5) Socio-technical shaping factors (3–4 min)

Q10. Domains and authenticity.

- *Main prompt:* How do you choose assignment domains/examples, and how do those choices affect engagement or performance?
- *Probes:* familiar vs unfamiliar domains; allowing domain choice; demotivating/excluding domain choices.

6) Closing (2–3 min)

Q11. Ideal supports / recommendations.

- *Main prompt:* If you could change one or two things to improve modelling learning in your context, what would you change?
- *Probes:* course design/scaffolding/assessment; feedback; tool features; “small change, big impact”.

Q12. Anything missing.

- *Main prompt:* Is there anything important I didn't ask but should have?

Close (say verbatim).

Thank you — this was very helpful. If you're interested, I can share a short summary of findings later, and I may follow up with one or two clarifying questions.

A.2 Student pilot interview guide (20–30 min)

0) Opening (1–2 min)

Script (say verbatim).

Hi! Thanks for taking the time. I'm doing a thesis on challenges in learning and doing software/conceptual modelling in higher education. This will take about 20–30 minutes. There are no right or wrong answers—I'm interested in your experience. With your permission, I'd like to audio-record so I don't miss details. The recording will be anonymised and used only for research. You can skip any question, and you can stop the interview at any point. Is it okay if I record?

1) Background and context (3–4 min)

Q1. Studies and modelling exposure.

- *Main prompt:* Can you tell me a bit about your studies and where modelling fits in (programme, year, courses)?
- *Probes:* which courses and purposes; notations used; modelling outside courses (work/internship/personal projects).

2) Experience with modelling (6–7 min)

Q2. Personal meaning of modelling.

- *Main prompt:* When you hear “software modelling” or “conceptual modelling”, what comes to your mind?
- *Probes:* first example/image; useful work vs extra documentation; 1–2 sentence explanation to a peer.

Q3. Useful moment story.

- *Main prompt:* Can you remember a moment when modelling felt particularly useful?
- *Probes:* context/task; what it helped (understand/decide/communicate); effect on confidence/motivation.

Q4. Frustrating moment story.

- *Main prompt:* Can you remember a moment when modelling felt confusing, pointless, or frustrating?
- *Probes:* cause (notation/tool/task/feedback/group/time); coping response; what would have helped.

3) Learning and challenges across modelling activities (7–8 min)

Q5. Learning a new notation/framework.

- *Main prompt:* When you learn a new notation/framework, what's usually the hardest part at the beginning?
- *Probes:* symbols vs semantics vs how to start; specific notation; strategies used to learn.

Q6. Creating a model from a text description.

- *Main prompt:* Imagine you get a text description and have to build a model. What steps do you take?
- *Probes:* starting point; structured strategy vs trial-and-error; what to include/omit; whether taught a step-by-step approach.

Q7. Deciding whether a model is “good enough”.

- *Main prompt:* Once you have a model, how do you decide whether it's good enough or needs changes?
- *Probes:* criteria (completeness/consistency/readability/fit); quality rules/checklists; comparing with peers.

4) Socio-technical factors (6–8 min)

Q8. Tools.

- *Main prompt:* What tools have you used for modelling, and how has that been?
- *Probes:* liked/disliked tools and why; tool makes modelling harder/easier; whether tools helped catch mistakes.

Q9. Assignments, domains, authenticity, and inclusion.

- *Main prompt:* What kinds of domains/examples did you work with in modelling assignments?

- *Probes*: motivating vs alien domains; domain choice; gentle inclusion probe (if appropriate).

Q10. Feedback and iteration.

- *Main prompt*: How did feedback on modelling work in your courses?
- *Probes*: type/timing of feedback; what would have helped most; effect of group work on modelling.

5) Closing (3–4 min)

Q11. “Good modeller” competence.

- *Main prompt*: What should a good modeller be able to do?
- *Probes*: habits/abilities; soft skills vs technical; whether courses helped develop them.

Q12. Improvements and support needs.

- *Main prompt*: If you could change one or two things in how modelling is taught, what would you change?

Final open question.

- Is there anything important about learning or using modelling that we didn’t cover?

A.3 Mapping of interview questions to research questions

Table A.1 summarises how the main interview prompts operationalise the analytic focus of each research question. Probes are used flexibly to adapt to participant experience and time constraints.

Table A.1: Mapping of pilot interview guide questions to research questions.

Guide	Question (main prompt)	RQ1	RQ2	RQ3
Student	Q1 Background and modelling exposure			✓
Student	Q2 Meaning/perception of modelling	✓		✓
Student	Q3 Useful moment (incident)	✓	✓	✓
Student	Q4 Frustrating moment (incident)	✓	✓	✓
Student	Q5 Learning a new notation/framework	✓	✓	✓
Student	Q6 Creating a model from text (procedure)	✓	✓	
Student	Q7 Judging quality / “good enough”	✓	✓	
Student	Q8 Tools	✓	✓	✓
Student	Q9 Domains/authenticity/inclusion		✓	✓
Student	Q10 Feedback and iteration	✓	✓	✓
Student	Q11 “Good modeller” competence	✓		✓
Student	Q12 Improvements / supports		✓	✓
Instructor	Q1 Teaching context and where modelling fits			✓
Instructor	Q2 Intended modelling competence	✓		✓
Instructor	Q3 Scaffolding from problem to model	✓	✓	✓
Instructor	Q4 Assessment focus and criteria	✓	✓	✓
Instructor	Q5 Recurring student challenges	✓		✓
Instructor	Q6 Concrete incident walkthrough	✓	✓	✓
Instructor	Q7 Instructor interventions and what works		✓	✓
Instructor	Q8 Tool supports and constraints		✓	✓
Instructor	Q9 Feedback and revision opportunities	✓	✓	✓
Instructor	Q10 Domains/authenticity effects		✓	✓
Instructor	Q11 Ideal supports / recommendations		✓	✓
Instructor	Q12 Anything missing (catch-all)	✓	✓	✓

ANALYTIC AUDIT TRAIL

This appendix reproduces a *selection* of analytical memos and open-code notes, not the full first-cycle corpus (277 codes, 78 memos). The selection is the memos named in Table 5.3 plus one code note per major cluster and the excerpts used in Table 4.5. Portuguese interviews are quoted from the English working transcripts used in analysis. Identifying course titles were already generalised in the vault notes.

B.1 How to read the cards

Each **memo** card stores one analytic idea. Fields: identifier (CaseMn), case, linked open-code IDs, transcript turn IDs (T00n), the idea, and why it mattered for category development. Memos are not findings by themselves; categories in Chapter 5 rest on comparison across memos.

Each **code** card stores one open code: a short title, case, turn IDs, and a verbatim excerpt. Codes stay close to the participant account. Student evidence, lecturer interpretation of student behaviour, and design wishes were kept distinct in later category work (Chapter 4).

Turn identifiers mark substantive transcript turns. They are not timestamps and were not invented after the fact.

B.2 Index of reproduced memos

S9 is not used to illustrate CAT9, CAT13, or CAT14. Negative or boundary memos that also appear above include S5M3 (can start without waiting) and L4M6 (withholds solved examples).

Table B.1: Memos reproduced below, by overview category (same identifiers as Table 5.3).

CAT	RQ	Memos in this appendix
CAT1	RQ1	S3M1, S8M1, L1M1
CAT2	RQ1	S1M1, S3M3, S2M3
CAT12	RQ1	S6M2, S4M2, S10M5
CAT14	RQ1	S6M1, S10M1, S7M5
CAT3	RQ2	S1M2, S7M1, L4M6
CAT4	RQ2	S5M3, S7M1, S8M1
CAT5	RQ2	S1M5, S7M3, S10M3
CAT6	RQ2	S4M3, S8M2, S10M2
CAT13	RQ2	S4M5, S5M4, S10M4
CAT7	RQ3	S3M4, S7M2, L4M5
CAT8	RQ3	S1M4, S7M2, L1M5
CAT9	RQ3	S4M1, S7M3, S10M5
CAT10	RQ3	S7M5, S10M5, L4M3
CAT11	RQ3	L1M3, S7M6, S8M3

B.3 Memo cards

S3M1 — earlier BPMN contact unlocks MSP start

Case: S3. **Codes:** S3-C04, S3-C05, S3-C06, S3-C17. **Turns:** T013, T017, T019, T043.

Idea. S3 encountered BPMN in two brief earlier courses before MSP. That prior contact let him skip basic assembly and start exercises as soon as they appeared, while a peer without that history needed more kick-start help. Crash-course conditions (about one week of BPMN, then the course moved on) built familiarity but not depth; MSP then added specific symbols and terminology on top.

Why it matters. For STGT, repeated socio-technical exposure—notation plus tool plus short practical windows—changed what “starting to model” felt like in the main modelling course. Prior contact with the same tool/notation pair (Bizagi + BPMN) reduced dependence on waiting for a board example.

S8M1 — modelling arrived as a project requirement first

Case: S8. **Codes:** S8-C01, S8-C03, S8-C04. **Turns:** T003, T011, T013.

Idea. At ISEL, S8 first had to produce UML/RE diagrams in the bachelor’s project without knowing the notations; the master’s Software Engineering course was the first proper teaching. By then a homemade process was already in place: underline requirements, pencil sketch, pen. Teacher examples mainly taught notation, not the expected answer.

Why it matters. Starting without a taught path (CAT1) can happen as a capstone requirement, not only as a first lab freeze. Later examples still help, but they arrive after students have already invented a sketching routine (CAT4).

L1M1 — breaking the ice is the main teaching hurdle

Case: L1. **Codes:** L1-C03, L1-C04, L1-C07, L1-C11. **Turns:** T012, T014, T022, T024, T037.

Idea. L1 ranks “breaking the ice”—actually starting to apply modelling to solve a problem—as harder than absorbing semantics alone. He sees most students wait for the teacher to start the solution; only a minority begin independently. Early work involves many typical notation errors until students practise mapping requirements into a model.

Why it matters. Lecturer interpretation centres procedural start-up, not just concept difficulty. This is a pedagogy–culture–confidence bundle: students, teaching moves, and expected teacher role interlock before tool or assessment details dominate.

S1M1 — notation and tool overload block starting

Case: S1. **Codes:** S1-C07, S1-C08, S1-C12. **Turns:** T018, T020, T029, T051.

Idea. On the FreeCol project, dense notation and a feature-rich tool stacked together. Many symbol choices created doubt about the "proper" arrow or relation before any modelling could begin. StarUML's interface felt like too much at once for simple entity-to-box work.

Why it matters. STGT: the difficulty is not cognition alone but a socio-technical bundle—course task, UML notation, and StarUML options jointly raised the entry cost. Tool design and notation load shape whether students can start at all.

S3M3 — notation detail stalls otherwise smooth progress

Case: S3. **Codes:** S3-C10, S3-C11, S3-C12. **Turns:** T025, T027.

Idea. S3 describes BPMN as broadly intuitive until notation details appear: duplicate gateway symbols, expanded subprocesses, unfamiliar constructs, or visually dense diagrams where sequence flow is hard to track. Progress can feel fine until a new symbol or layout problem stops work and triggers re-questioning.

Why it matters. Difficulty is not only "learning BPMN" once—it recurs at notation boundaries and at the visual/tool layer when models grow. This aligns with a socio-technical view where symbol set, diagram layout, and reading complexity interact.

S2M3 — framework burden in course not elsewhere

Case: S2. **Codes:** S2-C08, S2-C09, S2-C30, S2-C07. **Turns:** T022, T024, T026, T103, T106.

Idea. Course modelling felt frustrating because unfamiliar frameworks had to be learned before drawing, and abstractions had to fit FreeCol's existing rules. At work and in the dissertation, informal intuitive diagrams sufficed; prescribed frameworks and their multiplicity are off-putting.

Why it matters. STGT: the same person experiences modelling differently across contexts—course (framework + legacy codebase) versus self-directed work (purpose-built sketches). Institutional notation requirements may deter practice even when the student values abstraction.

S6M2 — justification replaces the compile button

Case: S6. **Codes:** S6-C05, S6-C09, S6-C15. **Turns:** T013, T021, T037.

Idea. S6 experiences requirements modelling as prolonged ambiguity: there is no compile button, so correctness comes from walking through the model and explaining why each element exists. The course required justification for actors, dependencies, and goal-to-class mappings; vague briefs and role-play interviews showed requirements shift with who you ask. Initial stress from indefiniteness later read as intentional pedagogy.

Why it matters. STGT highlights how evaluation norms (justification, traceability, stakeholder interviews) constitute the "technical" environment alongside tools. Modelling discomfort is socio-epistemic—living without executable feedback—not merely unfamiliar syntax.

S4M2 — worked examples unlock reasoning and granularity

Case: S4. **Codes:** S4-C03, S4-C04, S4-C05, S4-C06. **Turns:** T009, T011, T013.

Idea. The student can start from text alone but feels less confident without a teacher example, especially when task style is ambiguous. Examples that explain relation choices and contrast too-detailed versus too-abstract diagrams teach process, not just final shapes. A personal quality checklist grew from draft feedback rather than an official rubric.

Why it matters. Pedagogical artifacts (explained examples, feedback loops) scaffold judgment about "good enough"—a teaching-tool-learner interplay central to STGT.

S10M5 — exclusive notation ownership split the system

Case: S10. **Codes:** S10-C01, S10-C12, S10-C13. **Turns:** T005, T047, T049, T051, T057.

Idea. Splitting BPMN / requirements / UML by person produced two versions of the same system. The group invented draft-plus-review and a short carry-over talk before the next model, because the defence asked people individually. Explainability became a practical quality test: if the three authors could not walk it, it was too loaded. Thesis work later dropped strict UML in favour of communicating the architecture.

Why it matters. Same split-by-notation risk S7 feared; S10 actually saw divergence, then built the peer-review fix S7 described. CAT9/CAT12: oral defence plus a real reader (thesis) pull models toward simpler, explainable diagrams rather than notation completeness.

S6M1 — multi-notation RE breaks at translation gaps

Case: S6. **Codes:** S6-C01, S6-C02, S6-C08, S6-C14. **Turns:** T007, T019, T035.

Idea. S6's hardest moment was translating an iStar rationale model into UML: the class diagram looked syntactically fine but lost goal traceability because nobody explained how much strategic content should survive. Each notation family (iStar, NFR, UML) demands a different mindset—organisational/political versus structural—and restarting is expected in a multi-family course, but the missing translation methodology is the real pain. S6 wants one continuous worked example from goals through NFR decisions to UML with documented trace links.

Why it matters. Socio-technical RE pedagogy is not only teaching separate notations but orchestrating transitions between them. Without explicit bridge procedures and integrated artefacts, students produce locally correct diagrams that fail globally as requirements models.

S10M1 — BPMN-to-UML breaks when treated as the same model

Case: S10. **Codes:** S10-C02, S10-C03, S10-C04, S10-C05, S10-C06. **Turns:** T007, T009, T013, T015, T017, T019, T021.

Idea. S10's stuck moment was not a missing symbol. A payments service was clear as an external BPMN pool; a too-direct translation into UML left no place for it (actor, class, component, or omit). The teacher's after-class point was that models answer different questions. Names and requirements carried; gateways, lanes, and tasks did not become classes. One-to-one mapping produced action-classes; reasoning had to restart even though domain knowledge stayed. Traceability was mentioned, not practised across models.

Why it matters. This densifies CAT14 at IST AMS beyond S7's syntax-friction account, in the same family as S6's lost iStar-to-UML trace: locally sensible diagrams fail at the bridge. S10 names the missing method in student language ("different questions"), not only as a later teaching wish.

S7M5 — new syntax at the switch; a real reader later

Case: S7. **Codes:** S7-C01, S7-C14, S7-C15. **Turns:** T005, T033, T039, T041.

Idea. S7 did not call modelling difficult. Confusion was duplicated teaching of the same idea (Databases vs class diagrams) and syntax when moving families (relation types). Once already modelling, the next family was easier to start. Coursework models felt like they only passed through a teacher who had seen millions; the thesis architecture for supervisor and defence became a priority. After graduation S7 modelled to explain ideas without worrying about notation rules, and would rather teaching stressed usefulness and fewer families.

Why it matters. CAT14 in this case is light syntax friction plus duplicated course notations, not S6's lost goal-to-UML trace. CAT10 is a clean contrast: same student, low-stakes lecturer audience versus a reader who needed the system explained. Informal workplace modelling is S7's own report, not a lecturer claim.

S1M2 — worked examples bridge text and diagram

Case: S1. **Codes:** S1-C13, S1-C23, S1-C15. **Turns:** T031, T033, T071, T073.

Idea. S1 learns text-to-diagram mapping by trial and error, starting from components then placing actors. Worked examples—"this description translates to this model"—are named as the most useful feedback form. Templates for common steps would also help.

Why it matters. Without explicit process teaching (see S1-C15), example-driven learning becomes the main scaffold. STGT: pedagogy (examples, templates) substitutes for missing procedural instruction and couples learner strategy to course feedback design.

S7M1 — board labs split starters from copiers

Case: S7. **Codes:** S7-C02, S7-C03, S7-C04, S7-C05. **Turns:** T009, T011, T013, T017.

Idea. IST practicals unlocked work with a simple board example, a class-built requirements list, then a student at the board while the teacher stayed quiet until a student debate finished. S7 started immediately to have something if randomly picked, and because not trying drew more teacher displeasure than a wrong attempt. Classmates often waited and copied the board.

Why it matters. The same lab ritual produces two student strategies: attack-to-survive versus wait-to-copy. Lecturer "students wait to start" claims can be true of the room without being true of every student.

L4M6 — active discovery pedagogy resists passivity

Case: L4. **Codes:** L4-C21, L4-C22, L4-C25, L4-C32. **Turns:** T046, T048, T055, T067.

Idea. The lecturer refuses to front-load solved examples, uses icebreakers so shy students cannot wait all semester, and builds solutions gradually from photographed board work while admitting not knowing everything. That contrasts with students overwhelmed by continuous evaluation who rush task-to-task without assimilating.

Why it matters. Teaching stance (discovery, openness, pacing) interacts with course load and classroom social format—relevant when shy students prefer private feedback elsewhere.

S5M3 — early sharing and self-initiated drafts replace waiting

Case: S5. **Codes:** S5-C04, S5-C07, S5-C14. **Turns:** T009, T017, T033.

Idea. S5 rarely waits for a teacher demo: they draft version one, share work-in-progress in lab and group chat, and treat peer feedback as the safety net. Early public sharing reduces anxiety because errors surface early—though S5 acknowledges this pressure could harm shy peers. They want more assessed weekly diagram milestones with peer comments to institutionalise what their team already does informally.

Why it matters. Socio-technical conditions (assessment timing, lab culture, chat channels) shape modelling pedagogy as much as notation teaching. A student who initiates drafts and shares early experiences a different course than one who waits for authoritative solutions.

S1M5 — end-only feedback lets errors accumulate

Case: S1. **Codes:** S1-C22, S1-C18. **Turns:** T065, T067, T069, T043.

Idea. Modelling feedback came mainly at the final project discussion, not iteratively. Problems piled up until they surfaced as "completely wrong" at the end. S1 cannot recall formal validation rules from the course.

Why it matters. STGT: feedback timing and assessment design shape learning loops. Without mid-course modelling checks or memorable quality criteria, students rely on self-judgment and peer/dissertation comparisons instead of course scaffolding.

S7M3 — graded then told: rushed teammate forced a full redo

Case: S7. **Codes:** S7-C09, S7-C10. **Turns:** T023, T025.

Idea. The two-phase AMS project gave feedback after a grade, with no resubmit of that delivery—only a chance to fix the next phase or the defence. S7's group then had to rebuild the first-phase system from scratch. A "get-it-done-fast" teammate did almost everything immediately; the two who wanted to plan inherited a rushed model and lost the usual benefit of splitting load.

Why it matters. Feedback timing and group tempo interact. Staged deliveries do not create a learning loop if the grade is already locked and a fast member can freeze a weak model before others work.

S10M3 — timed local rewrite taught a boundary self-check

Case: S10. **Codes:** S10-C09, S10-C10. **Turns:** T033, T035, T037, T039.

Idea. Phase feedback caught a mixed BPMN perspective: an extra pool for something inside the organisation, and message flows that should have been sequence flows. With time left before the next delivery, the group did a large local rewrite, not a full abandon. After that, S10 started asking who is external before drawing. Notation nags were memorised; conceptual comments changed later work.

Why it matters. Unlike S7's grade-then-feedback full redo, S10 had time to act and the error was structural, not a rushed teammate. CAT5 is not only "too late": when a loop exists, conceptual feedback can become a new start habit.

S4M3 — shy students need private feedback channels

Case: S4. **Codes:** S4-C07, S4-C08. **Turns:** T015, T017, T033.

Idea. Design ambiguity (multiplicity, class vs attribute) is resolved in office hours or email, rarely in front of the class. Public presentation of half-finished diagrams feels stressful; quick private feedback would enable earlier sharing. The student explicitly asks for complete worked examples in materials so preparation can happen privately before targeted questions.

Why it matters. Social format of courses—public vs private validation—shapes modelling participation as much as notation instruction.

S8M2 — ungraded practice makes checking optional

Case: S8. **Codes:** S8-C02, S8-C05, S8-C06, S8-C07. **Turns:** T007, T009, T015, T019, T021.

Idea. Board examples plus a practice list were not submitted or checked; leaving was allowed. S8 says classmates skipped teacher checks from lack of interest because the exercises did not count, not from shyness. Camunda and Archi were available and unused; draw.io was enough. In the individual project the teacher walked around and almost always asked for changes, but S8 remembers no full redo. Sharing was easy with close friends and with the teacher, not with course-group strangers.

Why it matters. Feedback access (CAT6) here is graded-work gated: initiative plus whether the task counts. Optional semantic tools do not get used when a drawing tool already satisfies “requirements covered.”

S10M2 — reserved starter: notebook yes, board no

Case: S10. **Codes:** S10-C07, S10-C08, S10-C14. **Turns:** T025, T027, T029, T031, T055.

Idea. S10 solved the exercise in the notebook, then compared with the board. Silence was not “I cannot start” and not mainly fear of being wrong; it was dislike of twenty people watching while still thinking. Small arrow-type doubts often stayed unresolved until a classmate, a later search, or the correction. Desk walk-arounds made showing work easy; raising a hand in front of the class did not.

Why it matters. Answers S7M1: this reserved IST student is a starter, not a copier. CAT6 here is public-format gating. The same lab that S7 used for teacher goodwill is, for S10, a place to stay quiet unless the teacher comes to the desk.

S4M5 — tool semantics and workplace informality diverge from class

Case: S4. **Codes:** S4-C01, S4-C12, S4-C13. **Turns:** T005, T023, T025, T031.

Idea. ChatGPT helps explain diagrams in plain language but is rejected for generating graded models. Semantic lab tools catch invalid UML; fast sketch tools allow mistakes at home. A short internship used whiteboard sketches and prototypes—not formal UML—surprising expectations built from coursework.

Why it matters. Tool affordances and workplace norms sit alongside classroom notation teaching; misalignment may reinforce “modelling feels pointless” when industry practice looks informal.

S5M4 — AI and live tools in the group modelling loop

Case: S5. **Codes:** S5-C10, S5-C11, S5-C12. **Turns:** T023, T025, T027.

Idea. S5 uses ChatGPT to brainstorm alternative diagrams and simplify overcomplicated sequence descriptions, then rewrites everything manually before submission. Miro's live co-editing lowered the barrier for group modelling; Visual Paradigm added stricter semantics before coding. They want semantic consistency checking ("this message does not match any operation") more than generative box-drawing. Team passivity, not domain excitement, determines whether modelling effort feels worthwhile.

Why it matters. Tools, AI assistants, and team social dynamics form a coupled loop: the same Miro board that enables coordination also needs manual sync when code outpaces diagrams. AI enters as a private brainstorming layer, not a submission shortcut—pointing toward checker-style support rather than replacement modelling.

S10M4 — AI as a distrusted classmate, not a homework generator

Case: S10. **Codes:** S10-C11, S10-C18. **Turns:** T041, T043, T045, T057, T059.

Idea. S10 used AI after a self-check: screenshot plus system context, small questions (external vs actor), what should carry from BPMN, which model next. Never "make the whole model." It unblocked small doubts and cut some teacher questions, at the cost of missing discussions that taught more.

Why it matters. First Portuguese student incident for CAT13, matching S4–S6 explain-not-generate. Here AI also stands in for the public-ask S10 avoids (CAT6).

S3M4 — Bizagi scaffolds vocabulary and syntax

Case: S3. **Codes:** S3-C14, S3-C15, S3-C16. **Turns:** T034, T036, T038, T040.

Idea. Bizagi Modeler supported learning through in-interface vocabulary, hover explanations, and workflow validity checks. S3 found it intuitive from first open and says it clarified flow distinctions. At scale, one-by-one construction and occasional save corruption made the same tool frustrating for thesis-sized diagrams.

Why it matters. Tool affordances are part of the teaching-learning system: they lower vocabulary and syntax entry costs for beginners but do not equally support productivity or reliability at larger model sizes. Tool choice is tied to course reuse across three BPMN contexts and thesis work.

S7M2 — labs are gut feeling; exams are going on faith

Case: S7. **Codes:** S7-C06, S7-C07, S7-C08. **Turns:** T019, T021, T035.

Idea. Students treated the teacher's pre-drawn board solution as the most correct answer even while the teacher called alternatives a fun fact. In practicals, "good enough" was gut feeling and a tick-off of a homemade requirements list. In tests there was no compile signal, so completeness was fear plus faith. An open-source checker was shown once and dropped because the interface created more problems than it solved; remembered tools were family-specific, weak UI, and mainly for pretty PDF export.

Why it matters. Assessment risk, not notation knowledge, changes how students judge models. Wanting a compile button (CAT7) can coexist with rejecting the actual checker when tool friction (CAT8) outweighs the check.

L4M5 — whiteboard compensates for missing integrated tools

Case: L4. **Codes:** L4-C16, L4-C26, L4-C27, L4-C28, L4-C29. **Turns:** T039, T057, T061, T062, T064.

Idea. Without affordable, integrated, semantics-aware tooling across goal-oriented and object-oriented families, the lecturer relies heavily on whiteboard discussion, manual consistency checks, and family-specific tools like piStar for iStar only. Switching tools mid-course creates friction; the top realistic improvement would be a lightweight validator that bridges paradigms.

Why it matters. Tool ecology (licenses, semantics, integration) directly constrains what can be taught, checked, and iterated in class—a core STGT pairing of technique and infrastructure.

S1M4 — simpler tools reduce option overwhelm

Case: S1. **Codes:** S1-C20, S1-C08. **Turns:** T049, T051, T053, T055.

Idea. StarUML made modelling harder through option overload; draw.io felt easier because fewer choices were available. Tool design is seen as capable of improving the modelling experience, separate from notation learning.

Why it matters. STGT links tool affordances to learner engagement: a professional-feeling tool can increase cognitive load for students who are not deeply invested in software engineering. Tool simplification is a socio-technical lever alongside teaching.

L1M5 — proper tools versus draw.io in teaching stance

Case: L1. **Codes:** L1-C17, L1-C18, L1-C20, L1-C21. **Turns:** T057, T059, T061, T063, T071, T073.

Idea. L1 steers students away from draw.io toward metamodel-backed tools (e.g. Visual Paradigm) but struggles with licensing, imperfect checking, and past bad tools he abandoned. Class time is too short for tool onboarding and troubleshooting. He enthusiastically endorses a hypothetical faculty-provided free tool with feedback, checklists, and next-step guidance—explicitly a design preference, not an implemented system.

Why it matters. Tool policy is part of teaching: what counts as "modelling" vs drawing, what errors are caught, and who pays (institution vs student). Lecturer ideal tool features partially overlap student reports (vocabulary help, syntax warnings) but add institutional provisioning.

S4M1 — group work can kill modelling when code wins

Case: S4. **Codes:** S4-C02, S4-C09, S4-C11. **Turns:** T007, T017, T021.

Idea. In a generic shop group project, teammates pushed Spring implementation while the student maintained the class diagram alone; dismissal as "paperwork" stopped refinement until the model no longer matched code. Modelling worked later when one person owned an interesting library design; early sharing in groups turned diagrams into arguments rather than improvements.

Why it matters. Socio-technical conditions—ownership split, domain interest, group dynamics—can matter more than UML notation knowledge for whether modelling persists.

L4M3 — domain freedom trades lecturer workload for fairness

Case: L4. **Codes:** L4-C06, L4-C07, L4-C08, L4-C11. **Turns:** T026, T027, T028, T033.

Idea. Letting groups choose or refine domains—scoped when too simple or complex—increases marking and feedback work but avoids imposing uninterested domains the lecturer remembers as unfair. Modelling is reserved for problems complex enough to need communication beyond a weekend build. Under semester overload, some students still ask for tighter "just tell me what to do" tasks.

Why it matters. Course conditions (freedom, fairness, workload, concurrent evaluations) co-shape whether modelling feels worth the effort for both teacher and learners.

L1M3 — missing theory class breaks slide-only catch-up

Case: L1. **Codes:** L1-C14, L1-C15, L1-C22. **Turns:** T047, T051, T077.

Idea. L1 reports students skipping theoretical sessions, then learning from slides alone, misinterpreting concepts, and arriving at practicals unprepared. He does not repeat theory in labs and resists making slides a full tutorial. He attributes absences partly to Portuguese student culture, heavy multi-course workload, and continuous evaluation pressure—not lack of interest in modelling alone.

Why it matters. Course structure assumes theory attendance; practicals "just use" concepts. Socio-organisational conditions (semester load, evaluation design) undermine that assumption and create a theory–practice gap independent of notation choice.

S7M6 — competing courses made modelling a task to despachar

Case: S7. **Codes:** S7-C16. **Turns:** T013, T035.

Idea. S7 called the modelling workload adequate, but a packed second-year semester plus low interest meant finishing those tasks fast to protect other courses. Drawing and modelling were not a strength or a focus. That condition sits beside the board-lab split: some students only copied, and S7 still started—partly for the teacher's good books.

Why it matters. CAT11 here is interest plus concurrent load, not an overloaded modelling course itself. Student "despachar" is not the same as lecturers' ECTS-scarcity story.

S8M3 — similar families confuse; paid work drains care

Case: S8. **Codes:** S8-C08, S8-C09. **Turns:** T023, T025, T027, T031.

Idea. Switching was hardest when diagrams looked alike (requirements vs class relations); an example beside the task made it fine, and knowing one family made the next easier. Models still felt like drawings next to more tangible code, so perfection mattered less than presence. A tutoring job in the master's, not competing courses, cut interest. S8 now doubts professional utility and would teach workplace benefits rather than more notation.

Why it matters. CAT14 here is lookalike-notation confusion, not S6's lost traceability. CAT11's competing load is a paid job, not ECTS stacking. The utility wish is a student interpretation, not evidence that modelling "will die."

B.4 Open-code exemplars

The following notes include the excerpts that later entered Table 4.5 (S1-C04, S1-C12, S1-C08, S2-C08, S2-C04) and one additional code per cluster used in Chapter 5.

S1-C04 — modelling useful up to a point then arbitrary

Case: S1. **Turns:** T008.

Quote. “Modelling is useful for defining how system components interact up to a certain point. After that, it can feel arbitrary and like extra documentation.”

S1-C12 — symbol choices create doubt at start

Case: S1. **Turns:** T029.

Quote. “The high-level concepts were fairly straightforward, but the symbols and notation complicated things.” “Is this the proper notation for this? Is this the proper arrow for that?” That made it harder to begin.”

S1-C08 — tool options overwhelming for simple relations

Case: S1. **Turns:** T020, T051.

Quote. “The tool had so many options that it became completely overwhelming to know how to make a simple entity-to-box relation.”

S1-C23 — learn modelling best from worked examples

Case: S1. **Turns:** T071.

Quote. “Examples are more useful for me because I learn by seeing things.” “This textual description translates to this model.”

S2-C08 — course difficulty from unfamiliar frameworks

Case: S2. **Turns:** T024.

Quote. “Definitely in my Software Engineering course, because I had to use frameworks with which I was unfamiliar. I had to learn them before I could start drawing.”

S2-C04 — diagrams simplify concepts mind cannot hold

Case: S2. **Turns:** T014.

Quote. “Some concepts may be really complex and you cannot fit them in your mind. Making a small diagram can simplify them for you and for others who want to understand your work.”

S3-C04 — earlier courses were shallow crash courses

Case: S3. **Turns:** T013.

Quote. “It was basically a crash course, not very detailed, and we did not go very deep into modelling.”

Note. Industrial Management and Virtual Enterprises gave brief BPMN contact.

S4-C02 — group devalued model and abandoned it for code

Case: S4. **Turns:** T007.

Quote. “one person said it was basically unnecessary paperwork”

Note. Others pushed Spring implementation; diagram drifted from code within a week.

S4-C08 — shy student prefers private over public model sharing**Case:** S4. **Turns:** T017.*Quote.* “Not very comfortable in public. I am shy, so presenting a half-finished diagram to the whole class is stressful.”*Note.* Email or after-class showing preferred over class exposure.**S5-C04 — drafts version one without waiting for demo****Case:** S5. **Turns:** T009.*Quote.* “I rarely wait for a teacher demo.”*Note.* self-initiated draft replaces waiting for teacher example.**S6-C01 — UML class diagram lost trace from goals****Case:** S6. **Turns:** T007.*Quote.* “our pair produced a class diagram that looked syntactically correct but lost the goals”*Note.* teaching assistant confirmed classes fine but goal trace missing.**S7-C09 — grade given before feedback so no resubmit****Case:** S7. **Turns:** T023.*Quote.* “The grade was given before the feedback.”*Note.* Two-phase project; comments meant to fix the next phase or the defence, not a resubmission of the graded delivery.**S7-C16 — heavy semester made modelling something to finish fast****Case:** S7. **Turns:** T013, T035.*Quote.* “because modelling was not something that interested me much, I tried to get those course tasks done as fast as possible so I could focus more on the others.”*Note.* Second year had too many courses; modelling felt among the more boring ones. Workload itself was called adequate.**S8-C01 — bachelor project required models without teaching them****Case:** S8. **Turns:** T003.*Quote.* “I had the idea that I had to use UML, requirements engineering, and so on, because it was in the project requirements, but I had no idea how to use those notations.”*Note.* Bachelor’s barely named modelling; master’s Software Engineering was the first proper introduction.**S10-C02 — too-direct BPMN-to-UML translation of an external party****Case:** S10. **Turns:** T007, T009, T011.*Quote.* “I was trying to do a translation that was too direct from BPMN.”*Note.* Payments service clear as an external BPMN pool; UML choice among actor, class, component, or omit was the stuck point.

S10-C11 — AI as screenshot checker not model generator

Case: S10. **Turns:** T041, T043, T045, T057, T059.

Quote. “I never asked “make me the model of this whole system.””

Note. Screenshot plus short context; asked what should carry from BPMN, not a direct transform. Second opinion after self-check.

S10-C12 — split-by-notation made two versions of the system

Case: S10. **Turns:** T047, T049.

Quote. “the models started to diverge. The person doing BPMN interpreted a rule one way and the person doing UML interpreted it another way.”

Note. Group invented a draft-plus-review split and a five-to-ten-minute carry-over huddle before the next model. Defence was individual.

BIBLIOGRAPHY

- [1] S. Adolph, W. Hall, and P. Kruchten. “Using Grounded Theory to Study the Experience of Software Development”. In: *IEEE Software* 28.4 (2011), pp. 24–27 (cit. on pp. 10, 21, 25, 28–31).
- [2] D. Bogdanova and M. Snoeck. “CaMeLOT: An Educational Framework for Conceptual Data Modelling”. In: *Information and Software Technology* 110 (2019), pp. 92–107. DOI: [10.1016/j.infsof.2019.02.006](https://doi.org/10.1016/j.infsof.2019.02.006) (cit. on pp. 1, 5, 9, 12, 13, 16, 17, 22, 24, 51).
- [3] D. Bogdanova and M. Snoeck. “Domain Modelling in Bloom: Deciphering How We Teach It”. In: *The Practice of Enterprise Modeling (PoEM 2017)*. Vol. 305. Lecture Notes in Business Information Processing. 2017, pp. 3–17. DOI: [10.1007/978-3-319-70241-4_1](https://doi.org/10.1007/978-3-319-70241-4_1) (cit. on pp. 1, 5, 6, 9, 12, 13, 16, 18, 22, 24, 51).
- [4] D. Bork. “A Framework for Teaching Conceptual Modeling and Metamodeling Based on Bloom’s Revised Taxonomy of Educational Objectives”. In: *Proceedings of the 52nd Hawaii International Conference on System Sciences (HICSS)*. 2019 (cit. on pp. 1, 5, 7, 9, 12, 13, 16, 18, 22, 24, 51).
- [5] R. A. Buchmann et al. “Conceptual Modelling in Education: A Position Paper”. In: *CEUR Workshop Proceedings*. 2019 (cit. on pp. 1, 4, 5, 22, 52).
- [6] J. Cámara et al. “On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML”. In: *Software and Systems Modeling* 22 (2023), pp. 781–793. DOI: [10.1007/s10270-023-01105-5](https://doi.org/10.1007/s10270-023-01105-5) (cit. on pp. 23, 24, 52).
- [7] CC2020 Task Force. *Computing Curricula 2020: Paradigms for Global Computing Education*. Association for Computing Machinery and IEEE Computer Society, 2020. DOI: [10.5555/3467967.C5641384](https://doi.org/10.5555/3467967.C5641384) (cit. on p. 12).
- [8] R. Chakraborty and G. Liebel. ““We Do Not Understand What It Says”: Studying Student Perceptions of Software Modeling”. In: *Proceedings of the 25th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings (MODELS-C)*. CEUR-WS, 2022. URL: <https://ceur-ws.org/Vol-3418/paper06.pdf> (cit. on pp. 1, 9, 22, 51).

- [9] K. Charmaz. *Constructing Grounded Theory: A Practical Guide through Qualitative Analysis*. London: Sage Publications, 2006 (cit. on p. 10).
- [10] I. Dey. *Grounding Grounded Theory: Guidelines for Qualitative Inquiry*. San Diego: Academic Press, 1999 (cit. on p. 31).
- [11] S. Foss, T. Urazova, and R. Lawrence. “Automatic Generation and Marking of UML Database Design Diagrams”. In: *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education*. Vol. 1. ACM, 2022, pp. 626–632. doi: [10.1145/3478431.3499376](https://doi.org/10.1145/3478431.3499376) (cit. on pp. 23, 24, 52).
- [12] B. G. Glaser and A. L. Strauss. *The Discovery of Grounded Theory: Strategies for Qualitative Research*. Chicago, IL: Aldine Publishing Company, 1967 (cit. on p. 10).
- [13] I. Graßl et al. “Domain Diversity, Motivation, Inclusion, and Feedback in Software Modelling Education”. Manuscript submitted to ACM FSE 2026 (SEET track). 2026 (cit. on pp. 1, 8, 9, 13, 14, 16, 20, 22, 24, 53).
- [14] R. W. Hasker and M. Rowe. “UMLint: Identifying Defects in UML Diagrams”. In: *2011 ASEE Annual Conference & Exposition*. 2011, pp. 22.1558.1–22.1558.14. doi: [10.18260/1-2--18929](https://doi.org/10.18260/1-2--18929) (cit. on pp. 23, 24, 52).
- [15] R. Hoda, J. Noble, and S. Marshall. “Socio-Technical Grounded Theory for Software Engineering”. In: *IEEE Transactions on Software Engineering* 44.12 (2018), pp. 1208–1226 (cit. on pp. 2, 11, 21, 25, 29, 31–33).
- [16] R. Holmes, M. Allen, and M. Craig. “Dimensions of Experientialism for Software Engineering Education”. In: *Proceedings of the 40th International Conference on Software Engineering: Software Engineering Education and Training Track (ICSE-SEET ’18)*. New York, NY, USA: ACM, 2018. doi: [10.1145/3183377.3183380](https://doi.org/10.1145/3183377.3183380) (cit. on pp. 9, 14–16, 20, 22, 24).
- [17] B. Kitchenham and S. L. Pfleeger. “Teaching Survey Research in Software Engineering”. In: *ACM SIGSOFT Software Engineering Notes* 33.6 (2008), pp. 1–10.
- [18] D. R. Krathwohl. “A Revision of Bloom’s Taxonomy: An Overview”. In: *Theory Into Practice* 41.4 (2002), pp. 212–218 (cit. on pp. 2, 6, 11, 12, 17, 31).
- [19] T. Kühne. “What is a Model?” In: *Language Engineering for Model-Driven Software Development*. Ed. by J. Bézivin and R. Heckel. Vol. 4101. Dagstuhl Seminar Proceedings (DagSemProc). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2005, pp. 1–10. doi: [10.4230/DagSemProc.04101.15](https://doi.org/10.4230/DagSemProc.04101.15) (cit. on p. 4).
- [20] T. Li et al. “Understanding the Challenges and Requirements for Facilitating iStar Learning: An Empirical Study with iStar Learners”. In: *Information and Software Technology* 187 (2025), p. 107839. doi: [10.1016/j.infsof.2025.107839](https://doi.org/10.1016/j.infsof.2025.107839) (cit. on pp. 1, 2, 7, 9, 18, 19, 22, 24, 25, 28, 51).

- [21] G. Liebel, O. Badreddin, and R. Heldal. “Model Driven Software Engineering in Education: A Multi-Case Study on Perception of Tools and UML”. In: *2017 IEEE 30th Conference on Software Engineering Education and Training (CSEET)*. 2017, pp. 124–133. DOI: [10.1109/CSEET.2017.29](https://doi.org/10.1109/CSEET.2017.29) (cit. on pp. 1, 2, 8, 9, 15, 16, 19, 22, 24, 52, 53).
- [22] G. Liebel et al. “Human factors in model-driven engineering: future research goals and initiatives for MDE”. In: *Software and Systems Modeling* 23 (2024), pp. 801–819. DOI: [10.1007/s10270-024-01188-8](https://doi.org/10.1007/s10270-024-01188-8) (cit. on pp. 8, 9, 16, 20, 22).
- [23] J. Linaker, P. Runeson, and B. Regnell. “Challenges in Survey Research”. In: *Empirical Software Engineering* 20.4 (2015), pp. 924–962.
- [24] J. M. Lourenço. *The NOVAthesis L^AT_EX Template User’s Manual*. NOVA University Lisbon. 2021. URL: <https://github.com/joaomlourenco/novathesis/raw/main/template.pdf>.
- [25] I. Maslov. “Towards Empirically Validated Process Modelling Education Using a BPMN Formalism”. In: *Research Challenges in Information Science (RCIS 2022)*. Vol. 446. Lecture Notes in Business Information Processing. Springer, 2022, pp. 803–810. DOI: [10.1007/978-3-031-05760-1_58](https://doi.org/10.1007/978-3-031-05760-1_58) (cit. on pp. 1, 7, 9, 19, 22, 24, 53).
- [26] I. Maslov, S. Poelmans, and K. Rosenthal. “Conceptual Modeling Education: A Bibliometric Literature Review and Avenues for Future Research”. In: *Business & Information Systems Engineering* (2025), pp. 1–33. DOI: [10.1007/s12599-025-00930-w](https://doi.org/10.1007/s12599-025-00930-w) (cit. on pp. 1, 2, 4–6, 17, 21, 24, 51, 53).
- [27] Y. Sedelmaier and D. Landes. “A Research Agenda for Identifying and Developing Required Competencies in Software Engineering”. In: *International Journal of Engineering Pedagogy (iJEP)* 3.2 (2013), pp. 30–35. DOI: [10.3991/ijep.v3i2.2448](https://doi.org/10.3991/ijep.v3i2.2448) (cit. on pp. 5, 8, 9, 13, 15, 16, 21, 22, 24).
- [28] K. Sikkel and M. Daneva. “Teaching consistency of UML specifications”. In: *2010 5th International Workshop on Requirements Engineering Education and Training*. 2010. DOI: [10.1109/REET.2010.5633113](https://doi.org/10.1109/REET.2010.5633113) (cit. on pp. 23, 24, 53).
- [29] W. Silva et al. “What are the differences between group and individual modeling when learning UML?” In: *Proceedings of the XXXII Brazilian Symposium on Software Engineering*. ACM, 2018, pp. 308–317. DOI: [10.1145/3266237.3266255](https://doi.org/10.1145/3266237.3266255) (cit. on pp. 23, 24, 53).
- [30] K.-J. Stol, P. Ralph, and B. Fitzgerald. “Grounded Theory in Software Engineering Research: A Critical Review and Guidelines”. In: *IEEE Transactions on Software Engineering* 42.12 (2016), pp. 1201–1222 (cit. on pp. 2, 10, 21, 25, 28–33).
- [31] A. Strauss and J. Corbin. *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory*. 2nd ed. Thousand Oaks, CA: Sage Publications, 1998 (cit. on p. 10).

- [32] C. Verbruggen and M. Snoeck. “Exploratory Study on Students’ Understanding of Multi-Perspective Modelling”. In: *Enterprise, Business-Process and Information Systems Modeling*. Vol. 450. Lecture Notes in Business Information Processing. Springer, 2022. DOI: [10.1007/978-3-031-07475-2_22](https://doi.org/10.1007/978-3-031-07475-2_22) (cit. on pp. 23, 24, 53).

